

IRIS – Indicadores Rodoviários Integrados de Segurança

Produto 05 – Instruções de atualização dos dados

Pedro Augusto Borges dos Santos

2026-05-28

Índice

Apresentação	7
Resumo	7
1 Introdução	8
1.1 Escopo Geral	8
1.2 Estrutura do documento	9
2 Estrutura	10
2.1 Controle de pacotes e dependências	10
2.2 Controle de <i>pipeline</i>	11
2.3 Dados brutos	11
2.4 Funções	11
2.5 Relatórios	11
2.6 Dashboard	12
3 Execução	13
3.1 Importação dos dados	13
3.1.1 import_cameras.R	13
3.1.1.1 arrange_cameras_df()	13
3.1.1.2 calc_pontos_fiscalizacao()	13
3.1.1.3 calc_pontos_fisc_uf()	14
3.1.1.4 calc_pontos_fisc_mun()	14
3.1.2 import_cnt.R	14
3.1.2.1 create_table_list()	14
3.1.2.2 get_raw_table()	14
3.1.2.3 cnt_transform_to_df()	15
3.1.2.4 arrange_cnt_df()	15
3.1.3 import_datasus_cnes.R	15
3.1.3.1 extract_cnes_path()	15
3.1.3.2 read_cnes_files()	16
3.1.3.3 arrange_cnes_df()	16
3.1.3.4 join_cnes_df()	16
3.1.4 import_datasus_sim.R	16
3.1.5 import_detrans.R	17
3.1.6 import_dnit.R	17
3.1.6.1 read_dnit_df()	17
3.1.6.2 arrange_dnit_df()	17
3.1.7 import_ibge_pib.R	18

3.1.8	import_ibge_pop.R	18
3.1.9	raw_osm.R	18
3.1.10	import_osm.R	18
3.1.11	import_renach.R	19
3.1.12	raw_renaest.R	19
3.1.13	import_renaest.R	19
3.1.13.1	arrange_renaest()	19
3.1.13.2	join_renaest()	20
3.1.14	import_renainf.R	20
3.1.14.1	download_renainf_2023()	20
3.1.14.2	arrange_renainf()	20
3.1.15	import_renavam.R	21
3.1.15.1	read_renavam_2011_2024()	21
3.1.15.2	arrange_old_renavam()	21
3.1.15.3	arrange_renavam_idade()	21
3.1.16	import_snt.R	21
3.1.16.1	read_snt()	21
3.1.16.2	arrange_municipios()	22
3.1.17	import_taxa_bilhao.R	22
3.1.18	import_uf.R	22
3.1.19	Documentação da função create_uf()	22
3.2	Cálculo dos indicadores	23
3.2.1	indicadores_pilar_1.R	23
3.2.1.1	calc_i1()	23
3.2.1.2	calc_i2()	23
3.2.1.3	arrange_i2()	24
3.2.1.4	calc_i3()	24
3.2.1.5	arrange_i4()	24
3.2.1.6	join_indicadores_pilar1()	25
3.2.2	indicadores_pilar_2.R	25
3.2.2.1	calc_indicadores_cnt()	25
3.2.2.2	calc_ii5()	26
3.2.2.3	join_indicadores_pilar2()	26
3.2.3	indicadores_pilar_3.R	26
3.2.3.1	calc_iii1()	26
3.2.3.2	calc_iii2()	27
3.2.3.3	calc_iii3()	27
3.2.3.4	calc_iii4()	28
3.2.3.5	join_indicadores_pilar3()	28
3.2.4	indicadores_pilar_4.R	28
3.2.4.1	filter_frota_motos()	28
3.2.4.2	join_iv1()	29
3.2.4.3	calc_iv1()	29
3.2.4.4	calc_frota_reduzida()	29
3.2.4.5	join_iv2()	30
3.2.4.6	calc_iv2()	30
3.2.4.7	filter_infracoes_iv3()	30
3.2.4.8	join_iv3()	31
3.2.4.9	calc_iv3()	31

3.2.4.10	join_iv4()	31
3.2.4.11	calc_iv4()	32
3.2.4.12	filter_infracoes_iv5()	32
3.2.4.13	join_iv5()	32
3.2.4.14	calc_iv5()	33
3.2.4.15	join_iv6()	33
3.2.4.16	calc_iv6()	34
3.2.4.17	join_iv7()	34
3.2.4.18	calc_iv7()	34
3.2.4.19	join_indicadores_pilar4()	35
3.2.5	indicadores_pilar_5.R	35
3.2.5.1	join_v()	35
3.2.5.2	calc_indicadores_pilar5()	36
3.2.6	indicadores_pilar_6.R	36
3.2.6.1	calc_frota_total_uf()	36
3.2.6.2	calc_inf_total()	37
3.2.6.3	join_vi1()	37
3.2.6.4	calc_vi1()	37
3.2.6.5	arrange_cameras()	37
3.2.6.6	join_cameras_frota()	38
3.2.6.7	calc_cameras_frota()	38
3.2.6.8	calc_extensao()	38
3.2.6.9	join_vi5()	38
3.2.6.10	calc_vi5()	39
3.2.6.11	arrange_indicadores_cameras()	39
3.2.6.12	calc_total_cameras()	39
3.2.6.13	calc_infracoes_vi6()	40
3.2.6.14	join_vi6()	40
3.2.6.15	calc_vi6()	40
3.2.6.16	join_indicadores_pilar6()	40
3.2.6.17	count_cameras_cidades()	41
3.2.6.18	join_vi7()	41
3.2.6.19	calc_vi7()	41
3.2.6.20	join_pontos_fisc()	42
3.2.6.21	calc_indicadores_pontos()	42
3.2.6.22	join_vi11()	42
3.2.6.23	calc_vi11()	42
3.2.6.24	join_vi12()	43
3.2.6.25	calc_vi12()	43
3.2.7	indicadores_resultado.R	43
3.2.7.1	join_01()	43
3.2.7.2	calc_01()	44
3.2.7.3	join_02()	44
3.2.7.4	calc_02()	44
3.2.7.5	calc_03()	44
3.2.8	indicadores_join.R	45
3.3	Aplicação da correlação de Spearman	45
3.3.1	calc_correlacao_spear()	45
3.3.2	plot_cor_matrix()	46

3.4	Cálculo dos índices compostos	46
3.4.1	scale_indicadores()	46
3.4.2	invert_indicadores()	46
3.4.3	arrange_wide_df()	47
3.4.4	calc_pca()	47
3.4.5	extract_pca_var()	47
3.4.6	extract_pca_values()	48
3.4.7	join_pc_values_var()	48
3.4.8	calc_indicador_composto()	48
3.4.9	arrange_df_composto()	49
3.4.10	calc_notas()	49
3.4.11	create_classificacao()	49
3.5	Criação de mapas	50
3.5.1	plot_indicadores_map()	50
3.5.2	join_indicadores_map()	50
3.5.3	plot_classificacao_map()	50
3.5.4	plot_media_map()	51
3.6	Criação de gráficos	51
3.6.1	plots.R	51
3.6.1.1	plot_pca()	51
3.6.1.2	plot_boxplot_notas()	52
3.6.1.3	plot_media_classificacao()	52
3.6.2	plot_dinamicos.R	53
3.6.2.1	make_class_anim_plot()	53
3.6.2.2	render_classificacao()	53
3.6.2.3	make_radar_anim_plot()	54
3.6.2.4	render_radar()	54
3.6.2.5	export_animations()	54
3.6.2.6	create_indicadores_map_png()	55
3.6.2.7	animate_pngs()	55
3.7	Criação de tabelas	56
3.7.1	tbl_method.R	56
3.7.1.1	make_tbl_ind_class()	56
3.7.1.2	make_desc_ind_tbl()	56
3.7.1.3	make_gt_desc()	56
3.7.1.4	make_fontes_tbl()	57
3.7.1.5	make_gt_cnt()	57
3.7.1.6	make_codinf_tbl()	58
3.7.1.7	make_gt_pilar2()	58
3.7.1.8	make_pilar5_tbl()	58
3.7.1.9	make_pilar5_gt()	59
3.7.1.10	make_pilar6_cameras_tbl()	59
3.7.1.11	make_pilar6_cameras_gt()	60
3.7.2	tbl_results.R	60
3.7.2.1	make_tbl_indicadores()	60
3.7.2.2	make_gt_indicadores()	61
3.7.2.3	make_eda_df()	61
3.7.2.4	make_eda_gt()	62
3.7.2.5	arrange_pca_stats()	62

3.7.2.6	arrange_pca_results()	63
3.7.2.7	make_pca_stats_gt()	63
3.7.2.8	make_pca_results_gt()	64
3.7.2.9	make_tbl_invertidos()	64
3.7.2.10	make_gt_invertidos()	65
3.7.2.11	make_gt_compostos()	65
3.7.2.12	make_gt_classificacao()	66
3.8	Renderização dos relatórios	66
3.8.1	display_results()	66
3.8.2	display_stats()	67
3.8.3	display_ind_names()	67
3.8.4	display_cor_results()	67
3.8.5	display_cor_table()	68
3.8.6	display_eda_table()	68
3.8.7	display_pca_direction()	68
3.8.8	display_pca_variance()	69
3.8.9	display_notas()	69
3.8.10	display_classificacao()	69
3.8.11	calc_qnt_ind()	70
3.8.12	calc_media_regiao()	70
3.8.13	display_class_regiao()	71
3.9	Renderização do dashboard	71
3.9.1	data.R	71
3.9.1.1	arrange_classificacao_data()	71
3.9.1.2	arrange_indicadores_data()	72
3.9.1.3	join_desc_data()	72
3.9.2	mapa.R	72
3.9.2.1	plot_classificacao_leaflet()	72
3.9.2.2	plot_leaflet_geral()	73
3.9.2.3	plot_indicadores_leaflet()	73
3.9.3	plots.R	73
3.9.3.1	plot_radar_class()	73
3.9.4	table.R	74
3.9.4.1	make_class_dt()	74
3.9.4.2	make_ind_dt()	74
3.9.4.3	make_results_gt()	74
3.9.4.4	make_gt_bench()	75
4	Publicação	76
4.1	Arquivos dos relatórios	76
4.2	Publicação do dashboard	76
	Referências	79

Lista de Figuras

4.1	Passo 01 - Criação do token	77
4.2	Passo 02 - Comando de conexão	77
4.3	Passo 03 - Botão de publicação	77
4.4	Passo 04 - Painel de publicação	78

Apresentação

IRIS – Indicadores Rodoviários Integrados de Segurança

Produto 05 – Instruções de atualização dos dados

Versão 01 – 2026-05-28

Resumo

O objetivo do Projeto IRIS - Indicadores Rodoviários Integrados de Segurança - é estabelecer indicadores de desempenho para diagnosticar o nível de segurança viária nas unidades da federação brasileira. Criaram-se indicadores classificados conforme os seis pilares do atual plano de ações do Pnatrans: (I) Gestão da Segurança no Trânsito; (II) Vias Seguras; (III) Segurança Veicular; (IV) Educação para o Trânsito; (V) Vigilância; Promoção da Saúde e Atendimento às Vítimas; e (VI) Normatização e Fiscalização. O estabelecimento desses indicadores irá permitir o exercício de *benchmarking* da segurança viária entre os estados, considerando as diferentes áreas da mobilidade segura. Esse documento contempla a documentação do código-fonte do projeto, contendo instruções de atualização dos indicadores e respectivos produtos (relatórios e dashboard).

Capítulo 1

Introdução

Este relatório compõe a documentação completa do Projeto IRIS - Indicadores Rodoviários Integrados de Segurança. O objetivo da documentação é apresentar todo o processo de execução do código-fonte, estrutura e forma de publicação dos resultados, a fim de subsidiar futuras consultas, correções e atualizações do projeto.

Aqui assume-se que o usuário dessa documentação já possua experiência na linguagem R (R Core Team, 2024) e seus principais pacotes, como o `{targets}`¹, `{renv}`² e o meta-pacote `{tidyverse}`³, assim, a documentação evitará descrições e explicações prolongadas desses pacotes e sobre programação em R em geral. Também assume-se um conhecimento prévio dos *frameworks* `{quarto}`⁴ e `{shiny}`⁵. Ao longo do documento, estarão referenciados alguns materiais adicionais dessas ferramentas, para o que o usuário possa buscar mais informações.

1.1 Escopo Geral

O passo inicial de execução do código é instalar os pacotes necessários. Esse controle de dependências é feito pelo `{renv}` (Ushey e Wickham, 2024). A versão do R utilizada para o desenvolvimento desse projeto foi a 4.4.0. Assim, não há garantia de que o projeto funcione em outras versões do R. Com o projeto aberto, é necessário instalar o `{renv}` e executar a instalação das dependências:

```
install.packages('renv')
renv::init()
renv::restore()
```

De forma geral, toda a execução do código é controlada pelo `{targets}` (Landau, 2021). A configuração e ordem de execução do projeto é administrada pelo arquivo `_targets.R`, que fica na raiz do projeto. O seguinte comando executa a *pipeline*:

```
targets::tar_make()
```

¹<https://books.ropensci.org/targets/>

²<https://rstudio.github.io/renv/articles/renv.html>

³<https://www.tidyverse.org>

⁴<https://quarto.org>

⁵<https://shiny.posit.co>

Os reports em `.qmd` carregam os pacotes necessários que estão armazenados pelo `{renv}`. Para isso, o usuário deve indicar o local de armazenamento do `{renv}`. O caminho `renv/library/macos/R-4.4/aarch64-apple-darwin20/` deve ser alterado nos documentos `.qmd` antes de executar o pipeline com o `{targets}`, de acordo com o sistema de quem estiver rodando o projeto.

O `{targets}` não compila o dashboard. Para isso, deve-se executar o seguinte comando do pacote `{shiny}` (Chang et al., 2024):

```
shiny::runApp("app")
```

1.2 Estrutura do documento

A documentação do projeto está dividida em três partes. O Capítulo 2 apresenta a estrutura do projeto, descrevendo os arquivos presentes no repositório. O Capítulo 3 apresenta o processo de execução, com a descrição de cada *script*. Por fim, o Capítulo 4 demonstra como hospedar o dashboard após a sua atualização e também demonstra como extrair os relatórios do projeto.

Capítulo 2

Estrutura

A lista a seguir apresenta a estrutura dos arquivos e pastas na raiz do repositório:

- `LICENSE`: Licença MIT do projeto.
- `R`: Pasta com todos os scripts em R.
- `README.md`: Documentação do repositório.
- `_targets`: Pasta que armazena todos os objetos gerados pelo `{targets}`. Não editar os arquivos dessa pasta.
- `_targets.R`: Script que controla a execução com o `{targets}`.
- `app`: Pasta que inclui os arquivos do dashboard.
- `data-raw`: Pasta com os dados brutos de entrada do projeto.
- `indicadores-seg-viaria.Rproj`: Arquivo de projeto do RStudio.
- `renv`: Pasta com os pacotes e controle do `{renv}`.
- `renv.lock`: Arquivo do `{renv}` que controla as versões do R e dos pacotes.
- `report`: Pasta com os projetos do Quarto Markdown que geram os relatórios em PDF e DOCX

2.1 Controle de pacotes e dependências

Os arquivos `renv` e `renv.lock` são utilizados pelo `{renv}` para gerenciar as dependências do projeto. De acordo com a própria documentação do `{renv}` (Ushey e Wickham, 2024):

“A biblioteca do projeto, `renv/library`, é uma biblioteca que contém todos os pacotes atualmente usados pelo seu projeto. Essa é a mágica principal que faz o `renv` funcionar: em vez de ter uma única biblioteca do seu computador/sistema contendo os pacotes usados em todos os projetos, o `renv` fornece uma biblioteca separada para cada projeto. Isso traz os benefícios do isolamento: projetos diferentes podem usar versões diferentes de pacotes e instalar, atualizar ou remover pacotes em um projeto sem afetar nenhum outro projeto.”

Assim, o usuário pode utilizar os comandos `renv::install()` para instalar novos pacotes, `renv::snapshot()` para registrar e atualizar as informações controladas pelo `renv.lock` e `renv::restore()` caso queira instalar todos os pacotes utilizados pelo projeto.

2.2 Controle de *pipeline*

A pasta `_targets` e o arquivo `_targets.R` são gerados pelo pacote `{targets}`. Não modificar de forma manual a pasta `_targets/`, editar apenas o `_targets.R` caso seja necessário mudar a ordem de execução do projeto ou adicionar/remover passos da execução.

2.3 Dados brutos

A pasta `data-raw` armazena os dados brutos de entrada do projeto. Cada subdiretório contém uma fonte distinta. Alguns arquivos não estão no repositório devido ao seu tamanho (ver Seção 3.1) ou por não necessitarem de arquivos salvos para a sua importação (download direto pelo código, *webscraping*, pacotes de dados ou APIs).

2.4 Funções

A pasta `R` contém todos os scripts em `R` utilizados na pipeline do projeto. Os scripts `import_*.R` contêm funções que carregam, processam e armazenam os dados brutos presentes em `data-raw`. Cada script é responsável pelo processamento de uma das fontes de dados utilizadas no trabalho.

Os scripts `indicadores_*.R` contêm as funções que calculam os indicadores de desempenho da segurança viária. Cada script é responsável por calcular os indicadores de seu respectivo pilar, com base nos dados processados pela importação. O script `indicadores_join.R` une todos os indicadores calculados em um só objeto.

O script `correlacao.R` contém as funções que calculam a correlação de Spearman entre os indicadores e que criam os correlogramas utilizados nos relatórios. Todo o processo de aplicação da Análise de Componentes Principais (ACP), cálculo dos índices compostos e estabelecimento da classificação dos estados foi elaborado com as funções presentes no script `compostos.R`.

Os scripts `tbl_*.R` são responsáveis por organizar os dados e resultados das tabelas que são apresentadas nos relatórios, tanto nas partes de metodologia quanto nas partes de resultados. Todos os mapas apresentados nos relatórios foram gerados por funções presentes no script `mapas.R`. A mesma organização foi feita com as funções que geram os gráficos do relatórios, que estão presentes em `plots.R`, e também com as funções que geram os gráficos dinâmicos (`plot_dinamicos.R`). As funções de `report_utils.R` auxiliam na renderização dos resultados em texto presente nos relatórios.

Por fim, os scripts `targets_*.R` contêm listas que controlam a ordem de execução pelo `{targets}`. Esses arquivos são importados diretamente pelo `_targets.R` na raiz do projeto.

2.5 Relatórios

A pasta `report` apresenta os projetos do Quarto Markdown para cada relatório do IRIS. Esses projetos estão organizados em pastas numeradas pela respectiva entrega:

- `report/01`: Relatório de metodologia
- `report/02`: Relatório de resultados
- `report/04`: Relatório final
- `report/05`: Instruções de atualização dos dados
- `report/07`: Slides de apresentação do projeto em PDF e PPTX.
- `report/08`: Infográficos dinâmicos gerados a partir de `R/plot_dinamicos.R`

As pastas 01, 02, 04 e 05 têm a seguinte estrutura:

- `index.qmd`: Principal arquivo do relatório, contém a apresentação e o resumo.
- `*.qmd`: Demais arquivos de markdown. Cada arquivo contém um capítulo do relatório
- `img/`: Pasta com imagens externas, não geradas por código.
- `docs/`: Pasta com o *output* da renderização, contendo arquivos PDF e DOCX
- `_quarto.yml`: Arquivo com a configuração e metadados do relatório
- `references.bib`: Arquivo BibTeX com as referências bibliográficas
- `apa.csl`: Arquivo CSL com a estilização e padronização das referências.

2.6 Dashboard

A pasta `app` apresenta os scripts e dados utilizados no dashboard do IRIS. A pasta tem a seguinte estrutura:

- `data`: Pasta com os dados em formato RDS (binário do R), utilizados no dashboard.
- `R`: scripts em R utilizados no dashboard
 - `data.R`: carrega os objetos do `{targets}` e salva em `data`
 - `mapa.R`: apresenta funções que criam os mapas utilizados
 - `plots.R`: apresenta funções que criam os gráficos
 - `table.R`: apresenta funções que criam as tabelas
- `www`: pasta com imagens externas, não geradas por código, e com arquivo `style.css` que controla o estilo de alguns elementos do painel (Não editar manualmente).
- `about.md`: arquivo em Markdown com o texto presente na página “Sobre”.
- `home.md`: arquivo em Markdown com o texto presente na página “Início”.
- `style.scss`: arquivo que controla o estilo de alguns elementos do menu de navegação do painel. Ao compilar o dashboard, esse arquivo é transformado para `www/styles.css`, que por sua vez é lido pelo dashboard.
- `app.R`: script com o código-fonte do dashboard.

Capítulo 3

Execução

Este capítulo descreve as funções de cada script utilizado no projeto.

3.1 Importação dos dados

3.1.1 `import_cameras.R`

3.1.1.1 `arrange_cameras_df()`

Descrição

Processa um `data.frame` bruto contendo informações sobre câmeras de fiscalização, limpando os nomes das colunas, filtrando registros de câmeras fixas aprovadas, agrupando os dados por unidade federativa (UF) e tipo, e calculando o número total de câmeras em cada grupo.

Parâmetros

- `raw_df`: Um `data.frame` bruto contendo informações sobre as câmeras.

Retorno

- Um `data.frame` com as colunas: `sigla_uf`, `estado`, `tipo` e `n_cameras`.

3.1.1.2 `calc_pontos_fiscalizacao()`

Descrição

Calcula o número de pontos de fiscalização únicos por município com base nas câmeras fixas aprovadas, retornando o total por UF, município e tipo.

Parâmetros

- `raw_cameras_df`: Um `data.frame` bruto contendo informações sobre as câmeras.

Retorno

- Um `data.frame` com as colunas: `sigla_uf`, `estado`, `municipio`, `tipo` e `pontos_fiscalizacao`.

3.1.1.3 `calc_pontos_fisc_uf()`

Descrição

Consolida os pontos de fiscalização por UF e tipo, somando os valores totais e retornando o resultado em formato largo, com cada tipo representado por uma coluna.

Parâmetros

- `df_pontos_fiscalizacao`: Um `data.frame` contendo informações sobre os pontos de fiscalização por município, gerado pela função `calc_pontos_fiscalizacao()`.

Retorno

- Um `data.frame` com as colunas: `sigla_uf` e uma coluna para cada tipo de ponto de fiscalização.

3.1.1.4 `calc_pontos_fisc_mun()`

Descrição

Combina informações de pontos de fiscalização com dados de municípios, destacando capitais, ao agregar os pontos por município e unir com dados externos que identificam capitais.

Parâmetros

- `df_pontos_fiscalizacao`: Um `data.frame` contendo informações sobre os pontos de fiscalização por município, gerado pela função `calc_pontos_fiscalizacao()`.
- `df_osm`: Um `data.frame` contendo informações sobre municípios, incluindo suas capitais.

Retorno

- Um `data.frame` com as colunas: `nome_uf`, `nome_mun` e `pontos_fiscalizacao`.

3.1.2 `import_cnt.R`

3.1.2.1 `create_table_list()`

Descrição

Cria uma lista contendo as associações entre os anos (2022 e 2023) e as tabelas correspondentes, categorizadas por estado geral, pavimento, sinalização e geometria.

Parâmetros

- Nenhum.

Retorno

- Uma lista com as associações de tabelas por ano e categoria.

3.1.2.2 `get_raw_table()`

Descrição

Extraí uma tabela bruta de texto de uma página específica, delimitada por padrões como “Ótimo” no início e “Distrito Federal” no final.

Parâmetros

- `raw_text`: Um vetor de texto contendo as páginas do relatório.
- `page_number`: O número da página de onde será extraída a tabela.

Retorno

- Um vetor de texto contendo as linhas da tabela extraída.

3.1.2.3 `cnt_transform_to_df()`

Descrição

Converte um vetor de texto representando uma tabela em um `data.frame`, separando as colunas com base em múltiplos espaços e atribuindo nomes padrão às colunas.

Parâmetros

- `raw_text`: Um vetor de texto contendo os dados brutos da tabela, com a primeira linha representando os cabeçalhos.

Retorno

- Um `data.frame` com as colunas: `uf`, `otimo`, `bom`, `regular`, `ruim`, `pessimo` e `total`.

3.1.2.4 `arrange_cnt_df()`

Descrição

Processa um `data.frame` contendo dados de avaliação por estado, ajustando valores numéricos, convertendo colunas específicas e adicionando o ano de referência, além de remover linhas agregadas como regiões e o total do Brasil.

Parâmetros

- `raw_df`: Um `data.frame` bruto contendo os dados de avaliação.
- `ano_input`: O ano de referência para os dados.

Retorno

- Um `data.frame` processado com as colunas ajustadas e os valores numéricos corrigidos.

3.1.3 `import_datasus_cnes.R`

3.1.3.1 `extract_cnes_path()`

Descrição

Localiza arquivos na pasta `data-raw/datasus_cnes` que correspondam a um padrão específico de nome, retornando seus caminhos completos.

Parâmetros

- `pattern`: Um padrão de nome de arquivo para busca (expressão regular).

Retorno

- Um vetor de caracteres contendo os caminhos completos dos arquivos que correspondem ao padrão.

3.1.3.2 `read_cnes_files()`

Descrição

Lê arquivos no formato CSV, assumindo codificação `latin1`, pulando as três primeiras linhas e carregando um máximo de 27 linhas de cada arquivo.

Parâmetros

- `paths`: Um vetor de caminhos completos para os arquivos a serem lidos.

Retorno

- Um `data.frame` contendo os dados lidos dos arquivos especificados.

3.1.3.3 `arrange_cnes_df()`

Descrição

Organiza um `data.frame` de dados do CNES, limpando os nomes das colunas, separando a unidade da federação em código e nome, e adicionando a coluna do ano.

Parâmetros

- `df`: Um `data.frame` bruto contendo os dados do CNES.

Retorno

- Um `data.frame` com as colunas organizadas e a coluna `ano` adicionada.

3.1.3.4 `join_cnes_df()`

Descrição

Une múltiplos `data.frames` de dados do CNES em uma única tabela consolidada, renomeando e selecionando as colunas principais.

Parâmetros

- `clean_list`: Uma lista de `data.frames` limpos e organizados.

Retorno

- Um `data.frame` consolidado com as colunas: `cod_uf`, `nome_uf`, `ano`, `n_leitos_nao_sus`, `n_leitos_sus`, `n_leitos` e `n_profissionais`.

3.1.4 `import_datasus_sim.R`

Descrição

Processa os dados do Sistema de Informações sobre Mortalidade (SIM) relacionados a óbitos por acidentes de trânsito em 2022, agrupando e contando o número de óbitos por unidade federativa (UF).

Parâmetros

- `pkg_df`: Um `data.frame` contendo os dados do SIM, normalmente chamado de `rtdeaths`, do pacote `{roadtrafficdeaths}`.

Retorno

- Um `data.frame` com as colunas: `cod_uf`, `nome_uf_ocor` e `obitos`, representando o código da UF, o nome da UF e o total de óbitos registrados.

3.1.5 `import_detrans.R`

Descrição

Cria um `data.frame` contendo a sigla das unidades federativas (UFs) do Brasil e suas respectivas notas médias atribuídas aos DETRANs.

Parâmetros

- Nenhum.

Retorno

- Um `data.frame` com as colunas:
 - `sigla_uf`: Sigla da unidade federativa.
 - `nota_media`: Nota média atribuída ao DETRAN de cada UF.

3.1.6 `import_dnit.R`

3.1.6.1 `read_dnit_df()`

Descrição

Lê uma planilha do DNIT, extraíndo dados de uma aba específica, ignorando as primeiras linhas e carregando apenas as 31 primeiras linhas úteis.

Parâmetros

- `path_list`: Caminho completo para o arquivo Excel a ser lido.

Retorno

- Um `data.frame` contendo os dados brutos extraídos da planilha.

3.1.6.2 `arrange_dnit_df()`

Descrição

Processa um `data.frame` bruto do DNIT, limpando os nomes das colunas, selecionando colunas relevantes, filtrando linhas específicas, calculando o total pavimentado por estado e corrigindo nomes de UFs quando necessário.

Parâmetros

- `raw_df`: Um `data.frame` bruto contendo os dados do DNIT.
- `ano_input`: O ano de referência para os dados.

Retorno

- Um `data.frame` com as colunas: `sigla_uf`, `nome_uf`, `pista_simples`, `pista_dupla`, `total_pavimentado` e `ano`.

3.1.7 `import_ibge_pib.R`

Descrição

Converte um `data.frame` de PIB do IBGE para o formato longo, transformando as colunas de anos em uma variável e renomeando a coluna de nome da UF.

Parâmetros

- `raw_df`: Um `data.frame` contendo os dados do PIB, onde a primeira coluna representa os nomes das UFs e as colunas subsequentes representam os valores do PIB por ano.

Retorno

- Um `data.frame` no formato longo com as colunas: `nome_uf`, `ano` e `pib`.

3.1.8 `import_ibge_pop.R`

Descrição

Organiza um `data.frame` de população do IBGE, separando a coluna de unidades da federação em código e nome da UF, e convertendo os dados de população por ano para o formato longo.

Parâmetros

- `raw_df`: Um `data.frame` contendo os dados de população, com as colunas `Unidade da Federação` e as colunas de 2018 a 2023 representando a população de cada ano.

Retorno

- Um `data.frame` no formato longo com as colunas: `cod_uf`, `nome_uf`, `ano` e `populacao`.

3.1.9 `raw_osm.R`

Descrição

Esse script não está incluído no *pipeline* do `{targets}`. Ele faz o download os dados do OpenStreetMap com as vias de cada capital brasileira e salva esses dados em um `geojson` para cada município. Esse script demanda um processamento mais prolongado, assim ele deve ser executado apenas uma vez.

3.1.10 `import_osm.R`

Descrição

Calcula a distância total das vias de uma cidade a partir de um arquivo de dados no formato `geojson`, e extrai o nome do município e do estado do caminho do arquivo.

Parâmetros

- `path`: Caminho para o arquivo `geojson` (em formato `sf`) contendo os dados de vias da cidade.

Retorno

- Um `data.frame` (tibble) com as colunas: `nome_mun`, `nome_uf` e `dist_vias`, representando o nome do município, o nome do estado e a distância total das vias na cidade, respectivamente.

3.1.11 `import_renach.R`

Descrição

Processa um `data.frame` de dados do RENACH, removendo valores ausentes, filtrando os dados para anos posteriores a 2018 e categorias de CNH específicas, e somando o número de condutores por estado, ano e categoria de CNH.

Parâmetros

- `pkg_df`: Um `data.frame` do pacote `{driversbr}` contendo os dados do RENACH, com as colunas `ano`, `categoria_cnh`, `uf` e `condutores`.

Retorno

- Um `data.frame` com as colunas: `uf`, `ano`, `categoria_cnh` e `n_condutores`, representando o estado, o ano, a categoria de CNH e o número total de condutores para cada categoria.

3.1.12 `raw_renaest.R`

Descrição

Esse script não está incluído no *pipeline* do `{targets}`. Ele carrega os dados do Renaest, filtra para o ano de 2023 e salva em `data-raw`, no formato parquet. Esse script deve ser executado apenas uma vez.

3.1.13 `import_renaest.R`

3.1.13.1 `arrange_renaest()`

Descrição

Processa os dados de sinistros e vítimas, convertendo a coluna de data para formato de texto, tratando valores ausentes e inválidos, e calculando o número de valores ausentes por linha e o número total de colunas. Em seguida, agrega esses dados por estado (UF da ocorrência).

Parâmetros

- `df`: Um `data.frame` contendo os dados dos sinistros ou vítimas, com as colunas `uf_acidente`, `data_acidente` e outras variáveis relacionadas.

Retorno

- Um `data.frame` com as colunas: `uf_acidente`, `na_count` (número total de valores ausentes) e `total_count` (total de colunas processadas).

3.1.13.2 `join_renaest()`

Descrição

Aplica a função de organização (`arrange_func`) a uma lista de `data.frames` de sinistros e vítimas, agrega os resultados por estado e calcula o número total de valores ausentes e totais para cada UF.

Parâmetros

- `df_list`: Uma lista de `data.frames` com os dados dos sinistros e vítimas.
- `arrange_func`: A função a ser aplicada para processar cada `data.frame` da lista (geralmente `arrange_renaest`).

Retorno

- Um `data.frame` consolidado, com as colunas: `uf_acidente`, `na_count` e `total_count`, representando o estado, o número de valores ausentes e o total de colunas para cada estado.

3.1.14 `import_renainf.R`

3.1.14.1 `download_renainf_2023()`

Descrição

Baixa os dados de infrações do RENAINF para 2023, filtrando os links de arquivos CSV da página especificada e lendo os dados de cada arquivo CSV correspondente.

Parâmetros

- `url`: URL da página que contém os links para os arquivos CSV de infrações.

Retorno

- Um `data.frame` com os dados das infrações, combinados a partir dos arquivos CSV de 2023.

3.1.14.2 `arrange_renainf()`

Descrição

Organiza os dados de infrações do RENAINF, limpando e unificando os códigos de infrações e somando a quantidade de infrações por estado e código de infração. Apenas os códigos de infração selecionados são mantidos no resultado.

Parâmetros

- `raw_df`: Um `data.frame` contendo os dados brutos das infrações, com as colunas `uf`, `cod_infracao_2`, `cod_infracao_3`, e `quantidade`.

Retorno

- Um `data.frame` contendo as colunas `uf`, `cod_infracao` e `quantidade`, com as infrações agrupadas por estado e código, e filtradas pelos códigos de infração de interesse.

3.1.15 `import_renavam.R`

3.1.15.1 `read_renavam_2011_2024()`

Descrição

Filtra e organiza os dados do RENAVAM para o mês de dezembro e para modalidades que não sejam “TOTAL”, removendo a coluna `mes` do `data.frame`.

Parâmetros

- `pkg_df`: Um `data.frame` do pacote `{fleetbr}` contendo os dados do RENAVAM.

Retorno

- Um `data.frame` contendo os dados filtrados para o mês de dezembro e modalidades específicas.

3.1.15.2 `arrange_old_renavam()`

Descrição

Organiza e limpa os dados do RENAVAM de anos anteriores, renomeando a coluna de unidades da federação, removendo regiões e totais, e transformando os dados em formato longo, com a criação de uma coluna `modal`. A função também padroniza os valores da coluna `modal` e adiciona uma coluna de ano.

Parâmetros

- `df_old_renavam`: Um `data.frame` contendo os dados históricos do RENAVAM.
- `ano_input`: O ano de referência para os dados a serem processados.

Retorno

- Um `data.frame` no formato longo, com as colunas: `uf`, `modal`, `frota` e `ano`, representando o estado, o tipo de veículo, a frota e o ano.

3.1.15.3 `arrange_renavam_idade()`

Descrição

Organiza os dados do RENAVAM sobre a idade dos veículos, agrupando os dados por unidade da federação e ano do modelo, e somando o número de veículos por grupo.

Parâmetros

- `df`: Um `data.frame` contendo os dados do RENAVAM relacionados à idade dos veículos.

Retorno

- Um `data.frame` com as colunas `uf`, `ano_modelo` e `n_veiculos`, representando o estado, o ano do modelo e o número total de veículos por grupo.

3.1.16 `import_snt.R`

3.1.16.1 `read_snt()`

Descrição

Lê os dados da tabela do site do Sistema Nacional de Trânsito (SNT), limpa os nomes das colunas, renomeia as variáveis de acordo com o contexto e filtra os dados para remover a linha com o estado “BRASIL”. Converte a coluna de municípios integrados para formato numérico.

Parâmetros

- `url_snt`: URL do site que contém a tabela de dados do Sistema Nacional de Trânsito (SNT).

Retorno

- Um `data.frame` com os dados da tabela, contendo as colunas `n_integrados` e `nome_uf`.

3.1.16.2 `arrange_municipios()`

Descrição

Organiza os dados sobre os municípios, contando o número de municípios por estado (UF), excluindo o “Distrito Federal” e convertendo os nomes dos estados para maiúsculas.

Parâmetros

- `raw_df`: Um `data.frame` contendo os dados dos municípios.

Retorno

- Um `data.frame` contendo as colunas `nome_uf` e `n_municipios`, onde `n_municipios` é o número de municípios por estado.

3.1.17 `import_taxa_bilhao.R`

Descrição

Cria um `data.frame` (tibble) contendo os estados brasileiros (UF) e suas respectivas taxas de óbitos por bilhão de quilômetros percorridos.

Retorno

- Um `data.frame` contendo as colunas `nome_uf` (nome do estado) e `taxa_bilhao` (taxa associada ao estado).

3.1.18 `import_uf.R`

3.1.19 Documentação da função `create_uf()`

Descrição

Cria um `data.frame` (tibble) contendo informações sobre os estados brasileiros a partir de um `data.frame` de municípios. O retorno inclui códigos e siglas dos estados, nome dos estados em formato normalizado e em maiúsculas, além de identificar a região do Brasil a qual cada estado pertence.

Parâmetros

- `df_municipios`: `data.frame` que deve conter as colunas `UF` (código do estado) e `Nome_UF` (nome completo do estado).

Retorno

- Um `data.frame` com as seguintes colunas:
 - `cod_uf`: código do estado (character).
 - `nome_uf`: nome do estado.
 - `sigla_uf`: sigla do estado.
 - `nome_uf_upper`: nome do estado em letras maiúsculas.
 - `nome_uf_sem_acento`: nome do estado sem acentos.
 - `regiao_uf`: região do estado no Brasil (Norte, Nordeste, Sudeste, Sul, Centro-Oeste).

3.2 Cálculo dos indicadores

3.2.1 indicadores_pilar_1.R

3.2.1.1 calc_i1()

Descrição

Calcula o indicador `i.1`, que representa a proporção de municípios integrados ao Sistema Nacional de Trânsito (SNT).

Parâmetros

- `df_snt`: `data.frame` contendo o número de municípios integrados (`n_integrados`) e o número total de municípios (`n_municipios`) por estado.

Retorno

Um `data.frame` com as colunas:

- `nome_uf`: nome do estado.
- `indicador`: sempre “i.1”.
- `valor`: proporção de municípios integrados ao SNT.

3.2.1.2 calc_i2()

Descrição

Calcula o indicador `i.2`, que representa a proporção de valores ausentes nos dados do `Renaest`.

Parâmetros

- `df_renaest`: `data.frame` contendo os campos `na_count` (número de valores ausentes) e `total_count` (total de valores) por estado.

Retorno

Um `data.frame` com as colunas:

- `sigla_uf`: sigla do estado.

- `indicador`: sempre “i.2”.
- `valor`: proporção de valores ausentes.

3.2.1.3 `arrange_i2()`

Descrição

Organiza o indicador `i.2`, incluindo informações adicionais de estados ausentes e associando códigos e nomes dos estados.

Parâmetros

- `raw_i2`: `data.frame` bruto com os valores calculados de `i.2`.
- `uf_df`: `data.frame` de estados (`create_uf()`).

Retorno

- Um `data.frame` com as colunas:
 - `cod_uf`: código do estado.
 - `nome_uf`: nome do estado.
 - `indicador`: sempre “i.2”.
 - `valor`: resultados do indicador.

3.2.1.4 `calc_i3()`

Descrição

Calcula o indicador `i.3`, que representa o PIB per capita por estado.

Parâmetros

- `pib_df`: `data.frame` com valores de PIB por estado e ano.
- `pop_df`: `data.frame` com valores de população por estado e ano.

Retorno

Um `data.frame` com as colunas:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: sempre “i.3”.
- `valor`: PIB per capita.

3.2.1.5 `arrange_i4()`

Descrição

Organiza o indicador `i.4`, que representa a nota média dos Detrans.

Parâmetros

- `df_detrans`: `data.frame` contendo a nota média (`nota_media`) dos Detrans por estado.

Retorno

Um `data.frame` com as colunas:

- `sigla_uf`: sigla do estado.
- `indicador`: sempre “i.4”.
- `valor`: nota média dos Detrans.

3.2.1.6 `join_indicadores_pilar1()`

Descrição

Combina todos os indicadores do Pilar 1 (`i.1`, `i.2`, `i.3`, `i.4`) em um único `data.frame`.

Parâmetros

- `df_i1`: `data.frame` do indicador `i.1`.
- `df_i2`: `data.frame` do indicador `i.2`.
- `df_i3`: `data.frame` do indicador `i.3`.
- `df_i4`: `data.frame` do indicador `i.4`.
- `df_uf`: `data.frame` com informações dos estados (`create_uf()`).

Retorno

Um `data.frame` combinado com as colunas:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: tipo do indicador.
- `valor`: valor do indicador.

3.2.2 `indicadores_pilar_2.R`

3.2.2.1 `calc_indicadores_cnt()`

Descrição

Calcula os indicadores de qualidade das rodovias a partir dos dados de avaliação do CNT (Confederação Nacional do Transporte).

Parâmetros

- `df_cnt`: `data.frame` contendo as colunas `ruim`, `pessimo` e `total` para cada estado.
- `input_indicador`: string com o código do indicador a ser atribuído (ex.: “ii.1”, “ii.2”).

Retorno

Um `data.frame` com as colunas:

- `nome_uf`: nome do estado.
- `indicador`: código do indicador.
- `valor`: proporção de rodovias avaliadas como “ruim” ou “péssimo”.

3.2.2.2 `calc_ii5()`

Descrição

Calcula o indicador `ii.5`, que representa a proporção de pistas duplas em relação ao total de pavimentação rodoviária, com base nos dados do DNIT.

Parâmetros

- `df_dnit`: `data.frame` contendo as colunas `ano`, `pista_dupla` e `total_pavimentado` para cada estado.

Retorno

Um `data.frame` com as colunas:

- `nome_uf`: nome do estado.
- `indicador`: sempre “ii.5”.
- `valor`: proporção de pistas duplas.

3.2.2.3 `join_indicadores_pilar2()`

Descrição

Combina os indicadores do Pilar 2 (`ii.1`, `ii.2`, `ii.3`, ..., `ii.5`) em um único `data.frame`.

Parâmetros

- `list_cnt`: lista de `data.frames` dos indicadores derivados do CNT (ex.: `ii.1`, `ii.2`).
- `df_ii5`: `data.frame` do indicador `ii.5`.
- `df_uf`: `data.frame` com informações dos estados (`create_uf()`).

Retorno

Um `data.frame` combinado com as colunas:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: código do indicador.
- `valor`: valor do indicador.

3.2.3 `indicadores_pilar_3.R`

3.2.3.1 `calc_iii1()`

Descrição

Calcula o indicador `iii.1`, que representa a proporção de motocicletas na frota.

Parâmetros

- `df_renavam`: `data.frame` contendo os dados da frota de veículos por estado e modalidade.

Retorno

`data.frame` com:

- `sigla_uf`: sigla do estado.
- `indicador`: “iii.1”.
- `valor`: proporção de motocicletas.

3.2.3.2 `calc_iii2()`

Descrição

Calcula o indicador `iii.2`, que mede a proporção de veículos com mais de 10 anos de idade na frota.

Parâmetros

- `df_renavam_idade`: `data.frame` com informações de idade da frota por estado.

Retorno

`data.frame` com:

- `nome_uf`: nome do estado.
- `indicador`: “iii.2”.
- `valor`: proporção de veículos acima de 10 anos.

3.2.3.3 `calc_iii3()`

Descrição

Calcula o indicador `iii.3`, que contém o percentual mínimo de veículos com Airbag/ABS na frota

Parâmetros

- `df_renavam`: `data.frame` com os dados de frota e ano por estado.

Retorno

`data.frame` com:

- `sigla_uf`: sigla do estado.
- `indicador`: “iii.3”.
- `valor`: proporção de veículos novos equipados com dispositivos de segurança.

3.2.3.4 `calc_iii4()`

Descrição

Calcula o indicador `iii.4`, que mede o percentual mínimo de veículos com ISOFIX na frota.

Parâmetros

- `df_renavam`: `data.frame` com os dados de frota e ano por estado.

Retorno

`data.frame` com:

- `sigla_uf`: sigla do estado.
- `indicador`: “iii.4”.
- `valor`: proporção de veículos novos equipados com dispositivos de segurança.

3.2.3.5 `join_indicadores_pilar3()`

Descrição

Combina os indicadores do Pilar 3 (`iii.1`, `iii.2`, `iii.3`, `iii.4`) em um único `data.frame` com informações completas sobre cada estado.

Parâmetros

- `df_iii1`: `data.frame` do indicador `iii.1`.
- `df_iii2`: `data.frame` do indicador `iii.2`.
- `df_iii3`: `data.frame` do indicador `iii.3`.
- `df_iii4`: `data.frame` do indicador `iii.4`.
- `df_uf`: `data.frame` com informações dos estados (`create_uf()`).

Retorno

`data.frame` combinado com as colunas:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: código do indicador.
- `valor`: valor do indicador.

3.2.4 `indicadores_pilar_4.R`

3.2.4.1 `filter_frota_motos()`

Descrição

Filtra e calcula o total da frota de motocicletas e motonetas por estado.

Parâmetros

- `df_renavam`: `data.frame` com dados da frota de veículos.

Retorno

`data.frame` com:

- `uf`: sigla do estado.
- `frota`: total de motocicletas e motonetas.

3.2.4.2 `join_iv1()`

Descrição

Une os dados do RENACH com os dados de frota de motocicletas e motonetas.

Parâmetros

- `df_renach`: `data.frame` com os dados do RENACH
- `df_motos`: `data.frame` com a frota de motocicletas e motonetas

Retorno

`data.frame` com:

- `uf`: sigla do estado.
- `frota`: frota de motocicletas e motonetas.
- `condutores`: quantidade de condutores das categorias A e AB.

3.2.4.3 `calc_iv1()`

Descrição

Calcula o indicador `iv.1`, que mede a proporção de condutores habilitados em relação à frota de motocicletas e motonetas.

Parâmetros

- `df_joined`: `data.frame` combinado de condutores e frota.

Retorno

`data.frame` com:

- `sigla_uf`: sigla do estado.
- `indicador`: “iv.1”.
- `valor`: proporção de condutores por frota.

3.2.4.4 `calc_frota_reduzida()`

Descrição

Remove os veículos das categorias “BONDE”, “CHASSI PLATAF”, “REBOQUE”, “SEMI-REBOQUE”, “SIDE-CAR”, “TRATOR ESTEI”, “TRATOR RODAS” dos dados do Renavam.

Parâmetros

- `df_renavam`: `data.frame` com os dados do Renavam

Retorno

`data.frame` com:

- `uf`: sigla do estado.
- `frota_reduzida`: quantidade de frota reduzida.

3.2.4.5 `join_iv2()`

Descrição

Une os dados de infrações com os dados de frota reduzida e filtra as infrações para considerar apenas a direção sob efeito de álcool.

Parâmetros

- `df_uf`: `data.frame` com as informações dos estados
- `df_renainf`: `data.frame` com dados do Renainf
- `df_frota_reduzida`: `data.frame` com a frota reduzida de veículos

3.2.4.6 `calc_iv2()`

Descrição

Calcula o indicador `iv.2` - taxa de infrações por consumo de bebidas alcoólicas a cada 10 mil veículos

Parâmetros

- `df_joined`: `data.frame` combinado de frota reduzida e infrações.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.2”.
- `valor`: infrações por frota.

3.2.4.7 `filter_infracoes_iv3()`

Descrição

Filtra e calcula o total de infrações relacionadas ao excesso de velocidade.

Parâmetros

- `df_renainf`: `data.frame` com dados de infrações.

Retorno

`data.frame` com:

- `uf`: sigla do estado.

- `infracoes`: total de infrações.

3.2.4.8 `join_iv3()`

Descrição

Une as tabelas da frota reduzida, informações dos estados e infrações relacionadas ao excesso de velocidade.

Parâmetros

- `df_frota_reduzida`: `data.frame` com a quantidade de frota reduzida
- `df_uf`: `data.frame` com as informações dos estados
- `df_infracoes_iv3`: `data.frame` com as infrações relacionadas a excesso de velocidade

Retorno

`data.frame` com:

- `uf`: sigla dos estados,
- `infracoes`: quantidade de infrações
- `frota_reduzida`: quantidade de frota reduzida

3.2.4.9 `calc_iv3()`

Descrição

Calcula o indicador `iv.3`, que mede infrações por excesso de velocidade por frota reduzida.

Parâmetros

- `df_joined`: `data.frame` combinado de frota reduzida e infrações de velocidade.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.3”.
- `valor`: infrações por frota.

3.2.4.10 `join_iv4()`

Descrição

A função `join_iv4` processa e integra dados de frota de veículos e infrações de trânsito por estado. Ela filtra os veículos de interesse para o ano de 2023, calcula a frota total por estado, e cruza esses dados com informações das unidades federativas e infrações específicas (código “5185”).

Parâmetros

- `df_renavam`: `data.frame` contendo informações sobre a frota de veículos.

- `df_renainf`: `data.frame` contendo informações sobre infrações de trânsito.
- `df_uf`: `data.frame` contendo informações das unidades federativas.

Retorno

Um `data.frame` contendo:

- `uf`: Sigla da unidade federativa.
- `frota`: Frota total de veículos filtrados para cada estado.
- `quantidade`: Quantidade de infrações do código “5185”

3.2.4.11 `calc_iv4()`

Descrição

Calcula o indicador `iv.4`, que mede infrações relacionadas ao não-uso do cinto de segurança.

Parâmetros

- `df_joined`: `data.frame` combinado de frota e infrações.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.4”.
- `valor`: infrações por frota.

3.2.4.12 `filter_infracoes_iv5()`

Descrição

Filtra e calcula o total de infrações relacionadas ao uso de capacete.

Parâmetros

- `df_renainf`: `data.frame` com dados de infrações.

Retorno

`data.frame` com:

- `uf`: sigla do estado.
- `infracoes`: total de infrações.

3.2.4.13 `join_iv5()`

Descrição

A função `join_iv5` integra os dados de infrações, frota de motos e unidades federativas. Ela cruza os dados de infrações com informações sobre as unidades federativas e a frota de motos, com base nas siglas das unidades federativas.

Parâmetros

- `df_infracoes_iv5`: `data.frame` contendo informações sobre infrações de trânsito.
- `df_frota_motos`: `data.frame` contendo informações sobre a frota de motos.
- `df_uf`: `data.frame` contendo informações das unidades federativas.

Retorno

Um `data.frame` contendo:

- Dados de infrações, unidades federativas e frota de motos, com base na correspondência das unidades federativas.

3.2.4.14 `calc_iv5()`

Descrição

Calcula o indicador `iv.5`, que mede infrações relacionadas ao não-uso de capacete por frota de motocicletas.

Parâmetros

- `df_joined`: `data.frame` combinado de frota de motos e infrações.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.5”.
- `valor`: infrações por frota.

3.2.4.15 `join_iv6()`

Descrição

A função `join_iv6` integra os dados de frota reduzida, infrações e unidades federativas. Ela cruza os dados de frota com as unidades federativas e as infrações relacionadas ao código “7366” (uso de celular).

Parâmetros

- `df_frota_reduzida`: `data.frame` contendo informações sobre a frota reduzida de veículos.
- `df_renainf`: `data.frame` contendo informações sobre infrações de trânsito.
- `df_uf`: `data.frame` contendo informações das unidades federativas.

Retorno

Um `data.frame` contendo:

- Dados de frota reduzida, unidades federativas e infrações relacionadas ao código “7366”.

3.2.4.16 `calc_iv6()`

Descrição

Calcula o indicador `iv.6`, que mede infrações relacionadas ao uso de celular durante a condução.

Parâmetros

- `df_joined`: `data.frame` combinado de frota reduzida e infrações.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.6”.
- `valor`: infrações por frota.

3.2.4.17 `join_iv7()`

Descrição

A função `join_iv7` processa e integra dados de frota de veículos e infrações de trânsito por estado. Ela filtra os veículos de interesse para o ano de 2023, calcula a frota total por estado, e cruza esses dados com informações das unidades federativas e infrações específicas (código “5193” - infrações por não utilizar dispositivos de retenção).

Parâmetros

- `df_renavam`: `data.frame` contendo informações sobre a frota de veículos.
- `df_renainf`: `data.frame` contendo informações sobre infrações de trânsito.
- `df_uf`: `data.frame` contendo informações das unidades federativas.

Retorno

Um `data.frame` contendo:

- `uf`: Sigla da unidade federativa.
- `frota`: Frota total de veículos filtrados para cada estado.
- `quantidade`: Infrações do código 5193

3.2.4.18 `calc_iv7()`

Descrição

Calcula o indicador `iv.7`, que mede infrações relacionadas ao não-uso de equipamentos de retenção

Parâmetros

- `df_joined`: `data.frame` combinado de frota de automóveis e infrações.

Retorno

`data.frame` com:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: “iv.7”.
- `valor`: infrações por frota.

3.2.4.19 `join_indicadores_pilar4()`

Descrição

Combina os indicadores do Pilar 4 (iv.1 a iv.7) em um único `data.frame`, adicionando as informações completas dos estados.

Parâmetros

- `df_iv1`: `data.frame` do indicador iv.1.
- `df_iv2`: `data.frame` do indicador iv.2.
- `df_iv3`: `data.frame` do indicador iv.3.
- `df_iv4`: `data.frame` do indicador iv.4.
- `df_iv5`: `data.frame` do indicador iv.5.
- `df_iv6`: `data.frame` do indicador iv.6.
- `df_iv7`: `data.frame` do indicador iv.7.
- `df_uf`: `data.frame` com informações dos estados.

Retorno

`data.frame` combinado com as colunas:

- `cod_uf`: código do estado.
- `nome_uf`: nome do estado.
- `indicador`: código do indicador.
- `valor`: valor do indicador.

3.2.5 `indicadores_pilar_5.R`

3.2.5.1 `join_v()`

Descrição

A função `join_v` integra dados de saúde e população por unidade federativa. Ela filtra os dados de 2023 dos dois dataframes, cruzando as informações de saúde do `df_datasus_cnes` com a população do `df_ibge_pop` para cada unidade federativa.

Parâmetros

- `df_datasus_cnes`: `data.frame` contendo informações sobre dados de saúde, incluindo a coluna `cod_uf`.

- `df_ibge_pop`: `data.frame` contendo informações sobre a população das unidades federativas, incluindo as colunas `cod_uf` e `populacao`.

Retorno

`data.frame` com:

- `cod_uf`: código do estado
- `nome_uf`: nome do estado
- `populacao`: população residente
- `n_profissionais`: quantidade de profissionais da saúde
- `n_leitos`: quantidade de leitos
- `n_leitos_sus`: quantidade de leitos SUS
- `n_leitos_nao_sus`: quantidade de leitos não-SUS

3.2.5.2 `calc_indicadores_pilar5()`

Descrição

A função `calc_indicadores_pilar5` calcula e organiza indicadores de saúde relacionados a profissionais, leitos, e leitos do SUS por unidade federativa. Ela gera quatro indicadores por 1.000 habitantes e reorganiza os dados para um formato mais adequado para análise.

Parâmetros

- `df_joined`: `data.frame` contendo dados combinados de população e dados do CNES

Retorno

Um `data.frame` contendo:

- `cod_uf`: Código da unidade federativa.
- `nome_uf`: Nome da unidade federativa.
- `indicador`: O nome do indicador calculado (`v.1`, `v.2`, `v.3`, `v.4`).
- `valor`: O valor do indicador calculado para cada unidade federativa.

3.2.6 `indicadores_pilar_6.R`

3.2.6.1 `calc_frota_total_uf()`

Descrição

Calcula o total de veículos registrados por UF no ano de 2023.

Parâmetros

- `df_renavam`: `data.frame` com informações da frota veicular, contendo colunas `ano`, `uf`, e `frota`.

Retorno

Um `data.frame` com as colunas `uf` e `frota_total`.

3.2.6.2 `calc_inf_total()`

Descrição

Calcula o número total de infrações registradas por UF.

Parâmetros

- `df_renainf`: `data.frame` contendo informações sobre infrações, com colunas `uf` e `quantidade`.

Retorno

Um `data.frame` com as colunas `uf` e `infracoes`.

3.2.6.3 `join_vi1()`

Descrição

Realiza o merge entre os `data.frame` de infrações, frota total e UFs, relacionando-os pelos nomes e siglas das UFs.

Parâmetros

- `df_frota_total`: `data.frame` com as colunas `uf` e `frota_total`.
- `df_infracoes`: `data.frame` com as colunas `uf` e `infracoes`.
- `df_uf`: `data.frame` com as colunas `nome_uf_sem_acento` e `sigla_uf`.

Retorno

Um `data.frame` combinado com informações de frota e infrações.

3.2.6.4 `calc_vi1()`

Descrição

Calcula o indicador `vi.1`, que representa o número de infrações por 10.000 veículos.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `infracoes` e `frota_total`.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.6.5 `arrange_cameras()`

Descrição

Organiza as informações de câmeras de fiscalização por tipo e calcula o total de câmeras.

Parâmetros

- `df_cameras`: `data.frame` com informações sobre câmeras de fiscalização, contendo colunas `estado`, `tipo`, e `n_cameras`.

Retorno

Um `data.frame` com colunas para os diferentes tipos de câmeras e o total de câmeras por UF.

3.2.6.6 `join_cameras_frota()`

Descrição

Realiza o merge entre os `data.frames` de frota e câmeras de fiscalização, relacionando-os pela sigla da UF.

Parâmetros

- `df_frota`: `data.frame` com as colunas `sigla_uf` e `frota_total`.
- `df_cameras_arranged`: `data.frame` contendo informações de câmeras organizadas, com a coluna `sigla_uf`.

Retorno

Um `data.frame` combinado com as informações de frota e câmeras de fiscalização.

3.2.6.7 `calc_cameras_frota()`

Descrição

Calcula os indicadores `vi.2`, `vi.3`, e `vi.4`.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `cameras_total`, `cameras_Rodovia`, `cameras_Via Urbana`, e `frota_total`.

Retorno

Um `data.frame` com os indicadores calculados e os valores para cada UF.

3.2.6.8 `calc_extensao()`

Descrição

Calcula a extensão pavimentada total por UF no ano de 2023.

Parâmetros

- `df_dnit`: `data.frame` com informações sobre a extensão pavimentada, contendo as colunas `nome_uf` e `total_pavimentado`.

Retorno

Um `data.frame` com as colunas `nome_uf` e `total_pavimentado`.

3.2.6.9 `join_vi5()`

Descrição

Realiza o merge entre os `data.frames` de extensão pavimentada e câmeras de fiscalização em rodovias.

Parâmetros

- `df_extensao`: `data.frame` com a coluna `nome_uf` e `total_pavimentado`.
- `df_cameras`: `data.frame` com informações sobre câmeras de fiscalização, com a coluna `estado` e `n_cameras`.

Retorno

Um `data.frame` combinado com as informações de extensão pavimentada e câmeras de rodovia.

3.2.6.10 `calc_vi5()`

Descrição

Calcula o indicador `vi.5`, que representa a quantidade de câmeras de rodovia por quilômetro pavimentado.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `n_cameras` e `total_pavimentado`.

Retorno

Um `data.frame` com as colunas `nome_uf`, `indicador`, e `valor`.

3.2.6.11 `arrange_indicadores_cameras()`

Descrição

Organiza os indicadores de câmeras para cada UF, substituindo valores ausentes por valores padrão.

Parâmetros

- `df_uf`: `data.frame` com as colunas `cod_uf`, `nome_uf`, e `sigla_uf`.
- `df_ind_cameras`: `data.frame` com os indicadores de câmeras.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`, com valores padrão preenchendo as ausências.

3.2.6.12 `calc_total_cameras()`

Descrição

Calcula o total de câmeras por UF.

Parâmetros

- `df_cameras`: `data.frame` com as colunas `sigla_uf` e `n_cameras`.

Retorno

Um `data.frame` com as colunas `sigla_uf` e `n_cameras`.

3.2.6.13 `calc_infracoes_vi6()`

Descrição

Calcula o número total de infrações específicas (códigos 7455, 7463, e 7471) por UF.

Parâmetros

- `df_renainf`: `data.frame` contendo informações sobre infrações, com as colunas `uf` e `quantidade`, e a coluna `cod_infracao`.

Retorno

Um `data.frame` com as colunas `uf` e `n_infracoes`.

3.2.6.14 `join_vi6()`

Descrição

Realiza o merge entre os `data.frames` de câmeras totais, UFs e infrações específicas, relacionando-os pelas siglas e nomes das UFs.

Parâmetros

- `df_total_cameras`: `data.frame` com as colunas `sigla_uf` e `n_cameras`.
- `df_uf`: `data.frame` com as colunas `sigla_uf` e `nome_uf_sem_acento`.
- `df_infracoes_vi6`: `data.frame` com as colunas `uf` e `n_infracoes`.

Retorno

Um `data.frame` combinado com as informações de câmeras, UFs e infrações específicas.

3.2.6.15 `calc_vi6()`

Descrição

Calcula o indicador `vi.6`, que representa a quantidade de infrações por câmera.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `n_infracoes` e `n_cameras`.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.6.16 `join_indicadores_pilar6()`

Descrição

Realiza o merge entre os indicadores dos pilares 6, incluindo os indicadores de câmeras e infrações, e retorna um `data.frame` combinado.

Parâmetros

- `df_vi1`: `data.frame` com os indicadores `vi.1`.
- `df_vi5`: `data.frame` com os indicadores `vi.5`.
- `df_vi6`: `data.frame` com os indicadores `vi.6`.

- `df_vi7`: `data.frame` com os indicadores `vi.7`.
- `df_uf`: `data.frame` com as informações sobre as UFs.
- `df_cameras`: `data.frame` com os indicadores `vi.2`, `vi.3`, `vi.4`.

Retorno

Um `data.frame` combinado com os indicadores dos pilares 6.

3.2.6.17 `count_cameras_cidades()`

Descrição

Conta o número de câmeras de fiscalização por cidade, considerando apenas medidores fixos com resultado aprovado.

Parâmetros

- `df_cameras_raw`: `data.frame` com as informações das câmeras de fiscalização, incluindo as colunas `estado`, `municipio`, `tipo_medidor`, e `ultimo_resultado`.

Retorno

Um `data.frame` com as colunas `estado`, `municipio`, e o número de câmeras por cidade.

3.2.6.18 `join_vi7()`

Descrição

Realiza o merge entre o `data.frame` de informações sobre câmeras nas cidades e as informações de distâncias das vias no OSM.

Parâmetros

- `df_osm`: `data.frame` com as distâncias das vias, contendo as colunas `nome_mun`, `nome_uf`, e `dist_vias`.
- `df_cameras_cidades`: `data.frame` com as informações sobre câmeras por cidade, com as colunas `estado`, `municipio`, e `n`.

Retorno

Um `data.frame` combinado com as informações de câmeras e distâncias das vias.

3.2.6.19 `calc_vi7()`

Descrição

Calcula o indicador `vi.7`, que representa a quantidade de câmeras por quilômetro de via nas cidades.

Parâmetros

- `df_joined_vi7`: `data.frame` contendo as colunas `cameras` e `dist_vias`.
- `df_uf`: `data.frame` contendo informações sobre as UFs, com a coluna `nome_uf`.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.6.20 `join_pontos_fisc()`

Descrição

Realiza o merge entre o `data.frame` de pontos de fiscalização por UF e o total de frota por UF, calculando o total de pontos de fiscalização nas rodovias e vias urbanas.

Parâmetros

- `df_pontos_fisc_uf`: `data.frame` com as colunas `sigla_uf`, `Rodovia`, e `Via Urbana`.
- `df_renavam_total_uf`: `data.frame` com as colunas `uf` e `frota_total`.

Retorno

Um `data.frame` com as colunas `sigla_uf`, `Rodovia`, `Via Urbana`, e `total_pontos`.

3.2.6.21 `calc_indicadores_pontos()`

Descrição

Calcula os indicadores `vi.08`, `vi.09`, e `vi.10`, que representam a quantidade de pontos de fiscalização por frota (total, rodovia e via urbana).

Parâmetros

- `df_joined_pontos_fisc_uf`: `data.frame` contendo as colunas `total_pontos`, `Rodovia`, `Via Urbana`, e `frota_total`.

Retorno

Um `data.frame` com as colunas `sigla_uf`, `indicador`, e `valor`.

3.2.6.22 `join_vi11()`

Descrição

Realiza o merge entre os dados de pontos de fiscalização por UF e os dados de extensão de rodovias pavimentadas, retornando o total de pontos de fiscalização nas rodovias e o total de pavimentado por UF.

Parâmetros

- `df_pontos_fisc_uf`: `data.frame` contendo a coluna `sigla_uf` e os pontos de fiscalização nas rodovias.
- `df_dnit`: `data.frame` contendo as colunas `sigla_uf` e `total_pavimentado`.

Retorno

Um `data.frame` com as colunas `sigla_uf`, `pontos_fisc` (pontuação de fiscalização nas rodovias) e `total_pavimentado`.

3.2.6.23 `calc_vi11()`

Descrição

Calcula o indicador `vi.11`, que representa a quantidade de pontos de fiscalização por quilômetro nas rodovias pavimentadas.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `pontos_fisc` e `total_pavimentado`.

Retorno

Um `data.frame` com as colunas `sigla_uf`, `indicador`, e `valor`.

3.2.6.24 `join_vi12()`

Descrição

Realiza o merge entre os dados de pontos de fiscalização por município e os dados de distâncias de vias (do OSM), associando o número de pontos de fiscalização e a distância total das vias por município.

Parâmetros

- `df_pontos_fisc_mun`: `data.frame` contendo a coluna `nome_mun` e `pontos_fiscalizacao`.
- `df_osm`: `data.frame` contendo as colunas `nome_mun` e `dist_vias`.

Retorno

Um `data.frame` com as colunas `nome_uf`, `nome_mun`, `pontos_fiscalizacao`, e `dist_vias`.

3.2.6.25 `calc_vi12()`

Descrição

Calcula o indicador `vi.12`, que representa a quantidade de pontos de fiscalização por quilômetro de via no município.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `pontos_fiscalizacao` e `dist_vias`.

Retorno

Um `data.frame` com as colunas `nome_uf`, `indicador`, e `valor`.

3.2.7 `indicadores_resultado.R`

3.2.7.1 `join_01()`

Descrição

Realiza o merge entre os dados de frota de veículos (2022), os dados do Datasus e as informações da UF, retornando o total de frota por UF junto com os dados de óbitos.

Parâmetros

- `df_renavam`: `data.frame` contendo a coluna `uf` e a frota de veículos por UF.
- `df_datasus_sim`: `data.frame` contendo a coluna `cod_uf` e os dados de óbitos por UF.
- `df_uf`: `data.frame` contendo a coluna `sigla_uf` e o nome da UF.

Retorno

Um `data.frame` com as colunas `uf`, `total_frota`, `nome_uf` e os dados de óbitos.

3.2.7.2 `calc_01()`

Descrição

Calcula o indicador 0.1, que representa a quantidade de óbitos por 10.000 veículos (frota total) em cada UF.

Parâmetros

- `joined_df`: `data.frame` contendo as colunas `obitos` e `total_frota`.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.7.3 `join_02()`

Descrição

Realiza o merge entre os dados de população do IBGE e os dados do Datasus para o ano de 2022, associando a população e os dados de óbitos por UF.

Parâmetros

- `df_ibge_pop`: `data.frame` contendo a coluna `cod_uf` e a população por UF.
- `df_datasus_sim`: `data.frame` contendo a coluna `cod_uf` e os dados de óbitos por UF.

Retorno

Um `data.frame` com as colunas `cod_uf`, `populacao` e os dados de óbitos.

3.2.7.4 `calc_02()`

Descrição

Calcula o indicador 0.2, que representa a quantidade de óbitos por 100.000 habitantes em cada UF.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `obitos` e `populacao`.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.7.5 `calc_03()`

Descrição

Organiza o indicador 0.3.

Parâmetros

- `df_uf`: `data.frame` contendo as colunas `nome_uf` e `cod_uf`.

- `df_taxa`: `data.frame` contendo a coluna `nome_uf` e a taxa de mortes por bilhão de km.

Retorno

Um `data.frame` com as colunas `cod_uf`, `nome_uf`, `indicador`, e `valor`.

3.2.8 indicadores_join.R

Descrição

Realiza a combinação de múltiplos `data.frames` referentes aos pilares 1 a 6 dos indicadores, organizando-os pela coluna `cod_uf` e `indicador`. Adicionalmente, adiciona uma coluna `regiao_uf` que classifica cada UF de acordo com sua região geográfica.

Parâmetros

- `df_pilar1`: `data.frame` contendo os indicadores do pilar 1.
- `df_pilar2`: `data.frame` contendo os indicadores do pilar 2.
- `df_pilar3`: `data.frame` contendo os indicadores do pilar 3.
- `df_pilar4`: `data.frame` contendo os indicadores do pilar 4.
- `df_pilar5`: `data.frame` contendo os indicadores do pilar 5.
- `df_pilar6`: `data.frame` contendo os indicadores do pilar 6.
- `df_resultado`: `data.frame` contendo o resultado final.

Retorno

Um `data.frame` com os dados de todos os pilares e o resultado, ordenados por `cod_uf` e `indicador`, e com a coluna adicional `regiao_uf` representando a região geográfica da UF.

3.3 Aplicação da correlação de Spearman

Funções presentes no script `correlacao.R`

3.3.1 calc_correlacao_spear()

Descrição

Calcula a matriz de correlação de Spearman para um conjunto de indicadores, retornando a correlação e os valores de p para cada par de variáveis. A função organiza os resultados de forma que possa ser facilmente utilizada em visualizações, como gráficos de correlação.

Parâmetros

- `df_indicadores`: `data.frame` contendo as colunas `cod_uf`, `indicador` e `valor`, onde `valor` é o valor do indicador para cada UF.

Retorno

Um `data.frame` contendo a correlação de Spearman (`cor`) e o p -valor (`p_value`) para cada par de variáveis (`var_a` e `var_b`). O resultado é organizado de forma a ser usado diretamente em visualizações, como gráficos de dispersão ou *heatmaps*, com valores de correlação apresentados apenas na parte superior da matriz.

3.3.2 `plot_cor_matrix()`

Descrição

Cria um gráfico de correlograma para visualizar a matriz de correlação, destacando as correlações significativas entre variáveis. A função permite filtrar as variáveis a serem exibidas e ajusta automaticamente os valores para tornar o gráfico mais legível.

Parâmetros

- `tbl_cor`: `data.frame` contendo a correlação entre as variáveis, com as colunas `var_a`, `var_b`, `cor` e `p_value`.
- `ind_vector`: Vetor opcional de indicadores a serem filtrados na matriz de correlação. Se `NULL`, exibe todos os indicadores.

Retorno

Um gráfico de correlograma no formato `ggplot`, onde cada célula representa a correlação entre duas variáveis. As correlações não significativas (valor de $p > 0.05$) não terão valor exibido no gráfico, e as correlações significativas serão destacadas conforme a intensidade da cor. O gráfico inclui legenda e subtítulo explicativo.

3.4 Cálculo dos índices compostos

Funções presentes no script `compostos.R`.

3.4.1 `scale_indicadores()`

Descrição

Padroniza os valores dos indicadores, aplicando a técnica de normalização (z -score) para cada indicador individualmente.

Parâmetros

- `df_indicadores`: `data.frame` contendo os indicadores com suas respectivas colunas `indicador` e `valor`.

Retorno

Um `data.frame` com uma nova coluna chamada `valor_padronizado`, representando os valores dos indicadores padronizados (z -score).

3.4.2 `invert_indicadores()`

Descrição

Inverte os valores padronizados de determinados indicadores, para que os valores mais altos representem uma pior situação, conforme uma lista predefinida de indicadores.

Parâmetros

- `df_scaled`: `data.frame` contendo os indicadores padronizados na coluna `valor_padronizado`.

Retorno

Um `data.frame` com a coluna `valor_padronizado` ajustada para os indicadores que precisam ser invertidos, mantendo os outros inalterados.

3.4.3 `arrange_wide_df()`

Descrição

Reorganiza os dados padronizados em um formato “wide”, ou seja, cria uma coluna para cada indicador, facilitando a visualização e comparação entre os valores padronizados dos diferentes indicadores para cada unidade da federação.

Parâmetros

- `df_scaled`: `data.frame` contendo os indicadores e seus valores padronizados.

Retorno

Um `data.frame` em formato “wide”, com os indicadores como colunas e os valores padronizados correspondentes.

3.4.4 `calc_pca()`

Descrição

Realiza a análise de componentes principais (PCA) para os indicadores, utilizando um subconjunto de colunas do `data.frame` que correspondem a um pilar específico.

Parâmetros

- `df_indicadores_wide`: `data.frame` no formato “wide”, com indicadores como colunas.
- `pilar`: Um `character` que define qual pilar de indicadores será utilizado para a PCA.

Retorno

O resultado da análise PCA, obtido com a função `prcomp`, que contém os componentes principais calculados.

3.4.5 `extract_pca_var()`

Descrição

Extraí as variâncias explicadas pelos componentes principais resultantes da PCA, selecionando aqueles que têm um valor maior que 1 no critério de autovalor.

Parâmetros

- `pca_results`: Objeto resultante de uma análise PCA, retornado pela função `prcomp`.

Retorno

Um `data.frame` contendo as variâncias explicadas pelos componentes principais, com as colunas renomeadas como `var_1`, `var_2`, etc., de acordo com o número de componentes com autovalor maior que 1.

3.4.6 `extract_pca_values()`

Descrição

Extraí os valores dos componentes principais resultantes da PCA, selecionando os componentes que têm autovalor maior que 1.

Parâmetros

- `pca_results`: Objeto resultante de uma análise PCA, retornado pela função `prcomp`.

Retorno

Um `data.frame` contendo os valores dos componentes principais, com colunas selecionadas conforme o critério do autovalor.

3.4.7 `join_pc_values_var()`

Descrição

Realiza a junção dos valores dos componentes principais e das variâncias explicadas com os dados de indicadores, retornando um `data.frame` contendo o nome da unidade federativa e os valores e variâncias dos componentes principais.

Parâmetros

- `df_indicadores_wide`: `data.frame` contendo os indicadores no formato “wide”, com colunas de indicadores.
- `pca_values`: `data.frame` contendo os valores dos componentes principais extraídos da PCA.
- `pca_var`: `data.frame` contendo as variâncias explicadas pelos componentes principais.

Retorno

Um `data.frame` contendo a coluna `nome_uf` dos indicadores e as colunas dos valores e variâncias dos componentes principais.

3.4.8 `calc_indicador_composto()`

Descrição

Calcula o indicador composto utilizando os componentes principais (PC1, PC2, PC3) com base nos sinais das variâncias explicadas, ajustando os valores de cada componente de acordo com o sinal de seu respectivo somatório. Retorna um `data.frame` com o indicador composto calculado para cada unidade federativa.

Parâmetros

- `df_joined_pc`: `data.frame` contendo os valores dos componentes principais (PC1, PC2, PC3, etc.), assim como as colunas de variáveis explicativas (`var_PC1`, `var_PC2`, etc.) para cada unidade federativa.
- `pca_stats`: `data.frame` contendo as estatísticas da PCA, incluindo a variância explicada e os valores dos componentes principais.

Retorno

Um `data.frame` com as colunas `nome_uf`, os valores dos componentes principais (PC1, PC2, PC3) e o valor do indicador composto para cada unidade federativa.

3.4.9 `arrange_df_composto()`

Descrição

Adiciona uma coluna chamada `pilar` ao `data.frame` de indicadores compostos, associando o valor do pilar fornecido a cada linha.

Parâmetros

- `indicadores_compostos`: `data.frame` contendo os indicadores compostos, com colunas como `nome_uf` e `indicador_composto`.
- `pilar_input`: Valor do pilar a ser adicionado para cada linha.

Retorno

Um `data.frame` com a coluna adicional `pilar`.

3.4.10 `calc_notas()`

Descrição

Calcula a nota final para cada unidade federativa com base no indicador composto, utilizando uma escala de 0 a 10, normalizada pelo valor mínimo e máximo do indicador dentro de cada pilar.

Parâmetros

- `df_indicadores_compostos`: `data.frame` contendo os indicadores compostos, com colunas como `nome_uf`, `pilar`, e `indicador_composto`.

Retorno

Um `data.frame` com as colunas `nome_uf`, `pilar`, e `nota`, representando a nota final de cada unidade federativa para cada pilar.

3.4.11 `create_classificacao()`

Descrição

Cria a classificação dos indicadores compostos para cada unidade federativa, atribuindo uma classificação de 1 a 5 estrelas com base na nota calculada, utilizando o valor do quantil das notas dentro de cada pilar.

Parâmetros

- `df_indicadores_notas`: `data.frame` contendo as colunas `nome_uf`, `pilar`, `nota` e, após o cálculo, a coluna de classificação.

Retorno

Um `data.frame` com as colunas `nome_uf`, `pilar`, `nota`, `classificacao` (estrela de 1 a 5) e `classificacao_numeric` (valor numérico de 1 a 5).

3.5 Criação de mapas

Funções presentes no script `mapas.R`.

3.5.1 `plot_indicadores_map()`

Descrição

Gera um mapa temático de indicadores associados aos estados brasileiros, permitindo a visualização geográfica de cada indicador.

Parâmetros

- `sf_estados`: Objeto `sf` contendo os polígonos dos estados e sua geometria.
- `df_indicadores`: `data.frame` com as colunas `cod_uf`, `indicador`, e `valor`.
- `ind_input`: Indicador desejado a ser mapeado, passado como string.

Retorno

Objeto de plotagem `ggplot2` com o mapa do indicador escolhido.

3.5.2 `join_indicadores_map()`

Descrição

Combina múltiplos mapas temáticos de indicadores em layouts agrupados por pilares, organizando a visualização em uma lista de gráficos.

Parâmetros

- `list_map`: Lista de objetos `ggplot` representando mapas temáticos de indicadores. A ordem dos mapas na lista deve corresponder aos pilares e indicadores.

Retorno

Lista com os seguintes objetos de gráficos combinados:

- `pilar_i`: Mapas combinados para o Pilar I.
- `pilar_ii`: Mapas combinados para o Pilar II.
- `pilar_iii`: Mapas combinados para o Pilar III.
- `pilar_iv`: Mapas combinados para o Pilar IV.
- `pilar_v`: Mapas combinados para o Pilar V.
- `pilar_vi`: Mapas combinados para o Pilar VI.
- `resultado`: Mapas combinados para o pilar dos indicadores de resultado.

3.5.3 `plot_classificacao_map()`

Descrição

Gera um mapa temático exibindo a classificação dos estados brasileiros em um determinado pilar.

Parâmetros

- `df_classificacao`: `data.frame` contendo as colunas `nome_uf`, `pilar` e `classificacao`, com as classificações dos estados por pilar.
- `sf_estados`: Objeto `sf` com a geometria dos estados brasileiros e suas informações geográficas.
- `pilar_input`: Pilar selecionado para visualização, passado como `character`.

Retorno

Objeto do `ggplot2` apresentando o mapa de classificação dos estados para o pilar especificado.

3.5.4 `plot_media_map()`

Descrição

Gera um mapa temático exibindo a média da classificação numérica dos estados brasileiros em todos os pilares.

Parâmetros

- `df_class`: `data.frame` contendo as colunas `nome_uf` e `classificacao_numeric`, representando as classificações numéricas dos estados.
- `sf_estados`: Objeto `sf` com a geometria dos estados brasileiros e suas informações geográficas.

Retorno

Objeto do `ggplot2` apresentando o mapa com a média da classificação numérica de cada estado.

3.6 Criação de gráficos

3.6.1 `plots.R`

Funções presentes no script `plots.R`.

3.6.1.1 `plot_pca()`

Descrição

Gera gráficos de Análise de Componentes Principais (PCA) com base em resultados estatísticos e numéricos fornecidos, incluindo a representação das rotações de variáveis e a variação explicada por cada componente principal. Os gráficos permitem comparar as regiões de origem dos dados e visualizam componentes principais selecionados com base em autovalores.

Parâmetros

- `pca_stats`: `data.frame` contendo os resultados estatísticos da PCA, incluindo indicadores como autovalores (“eigenvalue”) e variância explicada por componente.
- `pca_numeric_results`: `data.frame` com os valores das observações projetadas nos componentes principais.

- `df_uf`: `data.frame` com informações complementares, incluindo os nomes e regiões associados a cada observação.

Retorno

Retorna um objeto `ggplot` com os gráficos gerados, contendo:

- Dispersão das observações nos componentes principais.
- Vetores que representam a rotação das variáveis originais.
- Linhas e labels para facilitar a interpretação visual dos resultados.

3.6.1.2 `plot_boxplot_notas()`

Descrição

Gera um *boxplot* para visualizar a distribuição das notas por unidade federativa, organizadas em ordem crescente da mediana das notas. As unidades federativas são coloridas com base na região correspondente.

Parâmetros

- `df_classificacao`: `data.frame` contendo os dados das notas, incluindo uma coluna `nota` e uma coluna `nome_uf` representando a unidade federativa.
- `df_uf`: `data.frame` contendo informações adicionais sobre as unidades federativas, incluindo as colunas `nome_uf` e `regiao_uf`.

Retorno

Retorna um objeto `ggplot` com o gráfico *boxplot*, apresentando:

- Distribuição das notas por unidade federativa.
- Coloração das unidades federativas baseada na região.
- Eixo x configurado para exibir valores com separador decimal em vírgula.

3.6.1.3 `plot_media_classificacao()`

Descrição

Gera um gráfico de barras horizontais que apresenta a média das classificações numéricas por unidade federativa. As barras são coloridas com base na região correspondente, e os valores médios são exibidos como rótulos próximos às barras.

Parâmetros

- `df_classificacao`: `data.frame` contendo as classificações numéricas (`classificacao_numeric`) e o nome da unidade federativa (`nome_uf`).
- `df_uf`: `data.frame` contendo informações adicionais sobre as unidades federativas, incluindo as colunas `nome_uf` e `regiao_uf`.

Retorno

Retorna um objeto `ggplot` com o gráfico gerado, apresentando:

- Média das classificações numéricas por unidade federativa.

- Coloração das barras baseada na região.
- Rótulos indicando os valores médios formatados com separador decimal em vírgula.

3.6.2 `plot_dinamicos.R`

3.6.2.1 `make_class_anim_plot()`

Descrição

Cria uma animação gráfica (`gganimate`) que exibe a classificação de estados brasileiros em diferentes pilares de segurança no trânsito. A função combina dados espaciais (`sf_estados`) com dados de classificação (`df_classificacao`) para gerar um mapa animado, onde cada quadro da animação representa um pilar específico.

Parâmetros

- `sf_estados`: Um objeto do tipo `sf` (Simple Features) contendo os polígonos dos estados brasileiros e suas informações espaciais.
 - Coluna obrigatória: `name_state` (nome dos estados em letras minúsculas).
 - Tipo: `sf` (data frame com geometria).
- `df_classificacao`: Um `data.frame` contendo a classificação dos estados para cada pilar.
 - Colunas obrigatórias:
 - * `nome_uf`: Nome dos estados em letras minúsculas.
 - * `pilar`: Identificação do pilar (ex: “Pilar I”, “Pilar II”, etc.).
 - * `classificacao`: Classificação associada ao pilar.
 - Tipo: `data.frame`.

Retorno

Retorna um objeto de animação (`gganimate`) que pode ser renderizado ou salvo como um arquivo de vídeo/GIF. A animação exibe um mapa dos estados brasileiros, onde a cor de cada estado representa sua classificação no pilar correspondente. Cada quadro da animação corresponde a um pilar específico.

3.6.2.2 `render_classificacao()`

Descrição

Renderiza uma animação gráfica (`gganimate`) em formato GIF, utilizando o renderizador `gifski`. A função é útil para salvar ou visualizar animações criadas com `gganimate` com alta qualidade e resolução.

Parâmetros

- `plot_anim`: Um objeto de animação criado com `gganimate`.
 - Tipo: `gganim` (objeto de animação do `gganimate`).

Retorno

Retorna a animação renderizada como um objeto do tipo `gif_image`, que pode ser salvo diretamente como um arquivo GIF ou exibido em um ambiente interativo.

3.6.2.3 `make_radar_anim_plot()`

Descrição

Cria uma animação gráfica (`gganimate`) no formato de gráfico de radar (ou polar) para visualizar as notas de classificação dos estados em diferentes pilares. A função utiliza um `data.frame` com dados de classificação para gerar um gráfico de barras em coordenadas polares, onde cada quadro da animação representa um estado diferente.

Parâmetros

- `df_classificacao`: Um `data.frame` contendo as notas de classificação dos estados para cada pilar.
 - Colunas obrigatórias:
 - * `nome_uf`: Nome do estado.
 - * `pilar`: Identificação do pilar (ex: “Pilar I”, “Pilar II”, etc.).
 - * `nota`: Nota associada ao pilar.
 - Tipo: `data.frame`.

Retorno

Retorna um objeto de animação (`gganimate`) que pode ser renderizado ou salvo como um arquivo de vídeo/GIF. A animação exibe um gráfico de radar onde as barras representam as notas dos pilares, e cada quadro da animação corresponde a um estado diferente.

3.6.2.4 `render_radar()`

Descrição

Renderiza uma animação gráfica (`gganimate`) no formato GIF, utilizando o renderizador `gifski`. A função é projetada para renderizar animações de gráficos de radar (ou polar) criadas com `gganimate`, garantindo alta qualidade e resolução.

Parâmetros

- `radar_animation`: Um objeto de animação criado com `gganimate`, geralmente produzido por funções como `make_radar_anim_plot()`.
 - Tipo: `gganim` (objeto de animação do `gganimate`).

Retorno

Retorna a animação renderizada como um objeto do tipo `gif_image`, que pode ser salvo diretamente como um arquivo GIF ou exibido em um ambiente interativo.

3.6.2.5 `export_animations()`

Descrição

Exporta uma lista de animações (`ganimate`) para arquivos GIF, salvando-as em um diretório especificado. A função utiliza `anim_save` do pacote `ganimate` para salvar cada animação com um nome correspondente da lista fornecida.

Parâmetros

- `list_anim`: Uma lista de objetos de animação criados com `ganimate`.
 - Tipo: `list` (contendo objetos `gganim`).
- `list_name`: Uma lista de nomes de arquivos (sem extensão) para salvar as animações.
 - Tipo: `list` ou `vector` de `character`.
- `path`: Caminho do diretório onde as animações serão salvas.
 - Tipo: `character`.

Retorno

A função não retorna um valor explícito. Ela salva as animações como arquivos GIF no diretório especificado, com os nomes fornecidos em `list_name`.

3.6.2.6 `create_indicadores_map_png()`

Descrição

Filtra e aplica um tema a uma lista de mapas de indicadores, retornando uma nova lista com os mapas selecionados e estilizados. A função é útil para preparar visualizações de mapas temáticos com um estilo consistente.

Parâmetros

- `map_indicadores`: Uma lista de mapas (objetos `ggplot`) representando indicadores.
 - Tipo: `list` (contendo objetos `ggplot`).
- `vector_select`: Um vetor de índices ou nomes que especifica quais mapas da lista devem ser selecionados.
 - Tipo: `vector` (numérico ou de caracteres).

Retorno

Retorna uma lista contendo os mapas selecionados, com o tema `theme_bw()` aplicado para estilização.

- Tipo: `list` (contendo objetos `ggplot` com o tema aplicado).

3.6.2.7 `animate_pngs()`

Descrição

Cria uma animação a partir de uma sequência de imagens PNG que correspondem a um padrão específico. A função lê as imagens de um diretório, combina-as em uma animação e ajusta a velocidade de exibição.

Parâmetros

- `png_pattern`: Um padrão de texto (regex) para filtrar os arquivos PNG no diretório especificado.
 - Tipo: `character`.

Retorno

Retorna uma animação criada a partir das imagens PNG correspondentes ao padrão fornecido.

- Tipo: Objeto `magick-image` (animação criada com o pacote `magick`).

3.7 Criação de tabelas

3.7.1 `tbl_method.R`

3.7.1.1 `make_tbl_ind_class()`

Descrição

Cria um `tibble` que organiza informações sobre os pilares, suas descrições e os indicadores associados a cada pilar de classificação.

Parâmetros

A função não recebe parâmetros.

Retorno

Retorna um `tibble` com três colunas:

- `pilar`: Identifica os pilares, incluindo um marcador para o resultado final.
- `descricao`: Descrição textual do conteúdo de cada pilar.
- `indicadores`: Indicadores associados a cada pilar, separados por vírgulas.

3.7.1.2 `make_desc_ind_tbl()`

Descrição

Cria um `tibble` que lista os indicadores utilizados, suas descrições, unidades de medida e fontes de dados.

Parâmetros

A função não recebe parâmetros.

Retorno

Retorna um `tibble` com as seguintes colunas:

- `indicador`: Nome do indicador, seguindo o formato “Pilar.Indicador”.
- `descricao`: Descrição textual do indicador.
- `unidade`: Unidade de medida do indicador.
- `fonte`: Fonte dos dados utilizados para calcular ou obter o indicador.

3.7.1.3 `make_gt_desc()`

Descrição

Cria uma tabela formatada usando o pacote `gt`, com informações filtradas por pilar de indicadores. A tabela exibe o indicador, sua descrição, unidade de medida e fonte de dados.

Parâmetros

- `tbl_desc`: `data.frame` contendo as colunas `indicador`, `descricao`, `unidade` e `fonte`.
- `ind_input`: `character` representando o pilar de indicadores (e.g., "I.", "II.") para filtrar os dados na tabela.

Retorno

Retorna um objeto `gt` contendo uma tabela formatada com:

- Colunas ajustadas para largura fixa.
- Títulos das colunas renomeados e estilizados.

3.7.1.4 `make_fontes_tbl()`

Descrição

Cria um `tibble` que lista os órgãos responsáveis pelos dados utilizados, o tipo de informação fornecida e o ano de referência dos dados.

Parâmetros

A função não recebe parâmetros.

Retorno

Retorna um `tibble` com as seguintes colunas:

- `orgao`: Nome do órgão responsável pelos dados.
- `informacao`: Tipo de informação fornecida pelo órgão.
- `ano_utilizado`: Ano ou intervalo de anos referente aos dados utilizados.

3.7.1.5 `make_gt_cnt()`

Descrição

Gera uma tabela formatada com os dados da pesquisa CNT de Rodovias, apresentando a classificação das condições das rodovias por unidade federativa.

Parâmetros

`cnt_df_2023`: Um `data.frame` contendo os dados da pesquisa CNT de 2023, com colunas correspondentes à unidade federativa (`uf`) e às categorias de avaliação das rodovias (ótimo, bom, regular, ruim, péssimo).

Retorno

Retorna um objeto `gt` que exibe os dados formatados, com:

- Rótulos para as colunas renomeados.
- Uma divisão de colunas sob o rótulo “Extensão (km)” para as categorias de avaliação.
- Números formatados com separador de milhares (.) e vírgula como separador decimal.
- Tamanho de fonte ajustado para “10pt”.

3.7.1.6 `make_codinf_tbl()`

Descrição

Cria uma tabela com os códigos de infração, seus fatores de risco associados e uma descrição detalhada da infração.

Parâmetros

Não possui parâmetros.

Retorno

Retorna um `data.frame` com as colunas:

- `cod_inf`: Código da infração.
- `fator_risco`: Categoria do fator de risco associado à infração.
- `descricao`: Descrição detalhada da infração, de acordo com a legislação de trânsito.

3.7.1.7 `make_gt_pilar2()`

Descrição

Gera uma tabela formatada utilizando o pacote `gt` para apresentar os dados do Pilar II, calculando o indicador como a proporção de extensão classificada como “ruim” ou “péssima”.

Parâmetros

- `df_cnt`: `data.frame` contendo os dados de classificação de extensão por UF, incluindo as colunas `otimo`, `bom`, `regular`, `ruim`, `pessimo`, `total`, e `ano`.
- `ind_label`: `character` definindo o rótulo do indicador calculado na tabela.

Retorno

Retorna um objeto `gt` com:

- As colunas ajustadas e renomeadas para descrever a extensão por qualidade (em quilômetros).
- O indicador calculado como proporção de “ruim” e “péssimo” sobre o total, formatado como percentual.
- Destaque visual em gradiente de cores na coluna do indicador calculado.

3.7.1.8 `make_pilar5_tbl()`

Descrição

Cria uma tabela para o Pilar V, calculando um indicador como a proporção de uma variável específica em relação à população, multiplicada por 1000.

Parâmetros

- `df_joined`: `data.frame` contendo os dados necessários, incluindo as colunas `nome_uf`, `populacao`, e a variável a ser utilizada como base do cálculo.

- `var`: `character` indicando o nome da variável do `df_joined` que será utilizada no cálculo do indicador.

Retorno

Retorna um `data.frame` contendo as colunas:

- `nome_uf`: Nome das UFs.
- A variável selecionada.
- `populacao`: População da UF.
- `indicador`: Resultados do indicadores

3.7.1.9 `make_pilar5_gt()`

Descrição

Gera uma tabela formatada em `gt` para o Pilar V, a partir de uma tabela com dados calculados, aplicando rótulos customizados e formatações para números.

Parâmetros

- `tbl_pilar5`: `data.frame` contendo os dados para o Pilar V, incluindo as colunas `nome_uf`, `populacao`, e `indicador`.
- `n_label`: `character` com o rótulo a ser usado para colunas que começam com `n_`.
- `ind_label`: `character` com o rótulo para a coluna `indicador`.

Retorno

Retorna um objeto `gt` formatado com as seguintes características:

- Nomes de colunas ajustados conforme os rótulos fornecidos.
- Formatação numérica com separador de milhares (ponto) e decimal (vírgula).
- Destaque visual da coluna `indicador` usando uma paleta de cores (Blues).
- Fonte ajustada para tamanho 10pt.

3.7.1.10 `make_pilar6_cameras_tbl()`

Descrição

Gera uma tabela para o Pilar VI, calculando um indicador relacionado ao número de câmeras por frota total e organizando os dados em ordem decrescente do valor calculado.

Parâmetros

- `df_joined`: `data.frame` contendo as colunas `sigla_uf`, `frota_total`, e a variável indicada por `var`.
- `var`: `character` representando o nome da coluna que será usada para calcular o indicador.

Retorno

Retorna um `data.frame` contendo:

- `sigla_uf`: Sigla da unidade federativa.
- Coluna indicada por `var`: Valor original da variável utilizada.
- `frota_total`: Total da frota para cada UF.
- `valor`: Indicador calculado (câmeras por 10.000 veículos), em ordem decrescente.

3.7.1.11 `make_pilar6_cameras_gt()`

Descrição

Gera uma tabela formatada para o Pilar VI, exibindo o indicador calculado para câmeras de segurança por frota total, com opções de formatação e coloração.

Parâmetros

- `tbl_pilar6`: `data.frame` contendo as colunas `sigla_uf`, `frota_total`, e `valor` (indicador calculado de câmeras por frota).
- `n_label`: `character` que define o título a ser utilizado nas colunas de câmeras.
- `ind_label`: `character` que define o título para a coluna do indicador calculado.

Retorno

Retorna uma tabela `gt` formatada com:

- **UF**: Sigla da unidade federativa.
- **Frota de veículos**: Total de veículos por UF.
- **valor**: Indicador de câmeras por frota (em 10.000 veículos).

A tabela possui:

- Formatação de números com separador de milhar (ponto) e separador decimal (vírgula).
- Coloração condicional na coluna `valor` com a paleta “Blues”.
- Tamanho da fonte ajustado para “10pt”.

3.7.2 `tbl_results.R`

3.7.2.1 `make_tbl_indicadores()`

Descrição

Cria uma tabela organizada para um conjunto específico de indicadores, filtrando os dados com base no prefixo do indicador fornecido.

Parâmetros

- `df_indicadores`: `data.frame` contendo as colunas `indicador`, `cod_uf`, e `valor`.
- `ind_input`: `character` que define o prefixo do indicador para filtrar as linhas (exemplo: “I.1”, “II.2”, etc.).

Retorno

Retorna uma tabela no formato largo (`pivot_wider`), onde cada indicador vira uma coluna, e o valor correspondente é atribuído. A coluna `cod_uf` é removida.

A tabela resultante contém:

- Uma linha para cada unidade federativa (UF).
- Colunas representando cada indicador específico do `ind_input`.

3.7.2.2 `make_gt_indicadores()`

Descrição

Cria uma tabela formatada para exibir os indicadores agrupados por região da unidade federativa, com as colunas formatadas de maneira personalizada.

Parâmetros

- `tbl_indicadores`: `data.frame` contendo dados de indicadores organizados por região e unidade federativa.

Retorno

Retorna uma tabela utilizando o pacote `gt` com as seguintes características:

- Agrupamento por `regiao_uf`.
- Colunas com nome de unidades federativas (`nome_uf`) e indicadores formatados.
- Nomes de indicadores convertidos para maiúsculas.
- Formatação numérica com 2 casas decimais e separadores de milhar e decimal customizados.
- Coloração de células nos valores dos indicadores usando uma paleta de cores **Blues**.

3.7.2.3 `make_eda_df()`

Descrição

Cria um `data.frame` para análise exploratória de dados (EDA) com estatísticas descritivas de cada indicador agrupado por região e indicador.

Parâmetros

- `df_indicadores`: `data.frame` contendo os dados dos indicadores com as colunas `regiao_uf`, `indicador`, e `valor`.

Retorno

Retorna um `data.frame` com as seguintes colunas:

- `regiao_uf`: Região da unidade federativa.
- `indicador`: Nome do indicador.
- Estatísticas descritivas para o `valor`:
 - `media`: Média dos valores.
 - `mediana`: Mediana dos valores.

- **desvio**: Desvio padrão dos valores.
- **min**: Valor mínimo.
- **max**: Valor máximo.
- Coluna **valor** contendo uma lista de valores para cada grupo (**regiao_uf**, **indicador**), após a agregação e cálculo das estatísticas descritivas.

3.7.2.4 `make_eda_gt()`

Descrição

Cria uma tabela formatada com estatísticas descritivas para cada indicador, com base no `data.frame` gerado pela função `make_eda_df`. A tabela é agrupada por região e apresenta várias estatísticas como média, desvio padrão, mínimo, máximo e mediana. A formatação é ajustada conforme o tipo de indicador.

Parâmetros

- **eda_df**: `data.frame` com as estatísticas descritivas geradas pela função `make_eda_df`.
- **ind_input**: Código do indicador (ex. “i.”, “ii.”, “iii.”) para ajustes de formatação específica de porcentagem.

Retorno

Retorna uma tabela **gt** com as seguintes características:

- As colunas representam as estatísticas (Média, Desvio padrão, Mínimo, Máximo, Mediana).
- As colunas de valores dos indicadores são formatadas com separador de milhar (ponto) e separador decimal (vírgula).
- Se o indicador for do tipo “i.”, “ii.” ou “iii.”, as colunas correspondentes são formatadas como percentuais.
- A tabela é agrupada por **regiao_uf** (região da unidade federativa) e cada indicador é mostrado com as estatísticas calculadas.

3.7.2.5 `arrange_pca_stats()`

Descrição

Organiza e prepara os resultados de uma análise de componentes principais (PCA). A função extrai os valores de rotação dos componentes principais, os autovalores (eigenvalues) e a variância explicada por cada componente. A tabela resultante apresenta as contribuições de cada variável (indicador) para cada componente, junto com os autovalores e a variância explicada.

Parâmetros

- **pca_results**: Resultados de uma análise PCA (geralmente, o objeto retornado por funções como `prcomp`).

Retorno

Retorna um `data.frame` com as seguintes informações:

- A tabela de rotação da PCA (as contribuições das variáveis para os componentes principais).
- A linha com os autovalores (eigenvalues) dos componentes principais.
- A linha com a proporção de variância explicada por cada componente.

Cada coluna corresponde a um componente principal (PC), e as linhas representam as variáveis originais, os autovalores e a variância explicada.

3.7.2.6 `arrange_pca_results()`

Descrição

A função `arrange_pca_results` combina os resultados de uma análise de componentes principais (PCA) com os dados originais, criando um `data.frame` que inclui as variáveis originais juntamente com as pontuações dos componentes principais para cada observação. O resultado facilita a visualização das projeções dos dados nos componentes principais.

Parâmetros

- `df_indicadores_wide`: Um `data.frame` com os dados originais, onde cada linha corresponde a uma observação (por exemplo, uma unidade federativa) e as colunas representam as variáveis dos indicadores.
- `pca_results`: Resultados de uma análise PCA, com a estrutura retornada pela função `prcomp`. O componente `pca_results$x` contém as pontuações dos componentes principais.

Retorno

Retorna um `data.frame` contendo:

- As colunas de `df_indicadores_wide` (presumivelmente incluindo uma coluna `nome_uf` que identifica as observações).
- As colunas que representam as pontuações dos componentes principais (PCs), que são extraídas do componente `x` de `pca_results`.

3.7.2.7 `make_pca_stats_gt()`

Descrição

A função `make_pca_stats_gt` formata e apresenta as estatísticas da análise de componentes principais (PCA) em uma tabela visual utilizando o pacote `{gt}`. Ela aplica formatação numérica, ajusta os rótulos das colunas e aplica uma paleta de cores para facilitar a interpretação dos resultados.

Parâmetros

- `pca_stats`: Um `data.frame` contendo as estatísticas da PCA, como autovalores e variância explicada. O `data.frame` deve ter uma coluna `indicador` e outras colunas com os valores das estatísticas.

Retorno

Retorna uma tabela `gt` com as seguintes características:

- Colunas de estatísticas formatadas com separadores de milhar e decimais ajustados.

- Coluna `indicador` com rótulos em maiúsculas, substituindo “EIGENVALUE” por “Autovalor” e “VARIANCIA” por “Variância”.
- A paleta de cores `RdBu` aplicada às estatísticas, com variação de cores dependendo dos valores, exceto para “Autovalor” e “Variância” que não recebem coloração.

3.7.2.8 `make_pca_results_gt()`

Descrição

A função `make_pca_results_gt` organiza e formata os resultados numéricos de uma análise de componentes principais (PCA) em uma tabela `gt`, agrupando os estados por região e ajustando a formatação numérica para facilitar a leitura dos dados.

Parâmetros

- `pca_numeric_results`: Um `data.frame` com os resultados numéricos da PCA. Este `data.frame` deve conter pelo menos uma coluna `nome_uf` para identificar os estados e suas respectivas variáveis numéricas associadas aos resultados da PCA.

Retorno

Retorna uma tabela `gt` com as seguintes características:

- Agrupamento por região geográfica (Norte, Nordeste, Sudeste, Sul, Centro-Oeste).
- Formatação de números com separadores de milhar e vírgula como separador decimal.
- Colunas com nomes de estados alterados para UF para exibição na tabela.
- Tamanho de fonte ajustado para 10pt na tabela.

3.7.2.9 `make_tbl_invertidos()`

Descrição

A função `make_tbl_invertidos` filtra e manipula dados de dois `data.frames` (`df_scaled` e `df_invertido`), criando uma tabela que combina indicadores invertidos com seus valores padronizados originais. Essa tabela é útil para análises que requerem a comparação entre os indicadores originais e seus inversos após o processo de padronização.

Parâmetros

- `df_scaled`: Um `data.frame` contendo os valores padronizados dos indicadores. Este deve ter colunas como `indicador`, `valor_padronizado`, e `nome_uf`.
- `df_invertido`: Um `data.frame` contendo os valores invertidos dos indicadores, com colunas como `indicador`, `valor_padronizado`, e `nome_uf`.

Retorno

Retorna um `data.frame` com as seguintes colunas:

- `regiao_uf`: Região do estado.
- `nome_uf`: Nome da unidade da federação (UF).
- `indicador`: O código do indicador.
- `invertido`: O valor invertido do indicador.

- **padronizado:** O valor original do indicador padronizado.

3.7.2.10 `make_gt_invertidos()`

Descrição

A função `make_gt_invertidos` cria uma tabela formatada utilizando o pacote `gt` a partir de um `data.frame` que contém indicadores invertidos e seus valores padronizados. Dependendo do valor de `ind_input`, a função cria uma tabela específica com colunas para indicadores padronizados e invertidos, organizando-os em grupos por região e formatando os valores numéricos.

Parâmetros

- **df_tbl_invertido:** Um `data.frame` contendo os valores invertidos e padronizados dos indicadores, com colunas como `nome_uf`, `indicador`, `invertido`, e `padronizado`.
- **ind_input:** Um valor string que determina quais indicadores serão selecionados para a tabela. Dependendo deste valor, diferentes conjuntos de indicadores são processados (ex: "i.", "ii.", "iii.", etc.).

Retorno

Retorna um objeto `gt` que exibe uma tabela formatada com:

- Colunas UF (nome da unidade da federação),
- Indicadores padronizados e invertidos, organizados por região (separados em grupos como “Padronizado” e “Invertido”).

3.7.2.11 `make_gt_compostos()`

Descrição

A função `make_gt_compostos` cria uma tabela formatada utilizando o pacote `gt` a partir de dois `data.frames`: um contendo indicadores compostos e outro contendo notas associadas. A tabela resultante apresenta os indicadores compostos e suas notas para uma região específica, filtrada pelo parâmetro `pilar_input`.

Parâmetros

- **df_indicadores_compostos:** Um `data.frame` com os indicadores compostos, incluindo colunas como `nome_uf`, `pilar`, e indicadores principais.
- **df_indicadores_notas:** Um `data.frame` com as notas associadas aos indicadores compostos, contendo colunas como `nome_uf`, `pilar`, e `nota`.
- **pilar_input:** Um valor que filtra os dados pelo pilar desejado.

Retorno

Retorna uma tabela `gt` com as seguintes características:

- Colunas UF, Índice Composto, e Nota.
- Indicadores compostos e suas respectivas notas.
- Valores formatados com 2 casas decimais, separador de milhar por ponto e separador decimal por vírgula.

- Coloração dos valores da coluna **Nota** com uma paleta de cores “Blues”.

3.7.2.12 `make_gt_classificacao()`

Descrição

A função `make_gt_classificacao` gera uma tabela formatada utilizando o pacote `gt`, com base em um `data.frame` contendo informações de classificação e notas. Ela permite visualizar a classificação e a nota de cada unidade da federação para um pilar específico, selecionado pelo parâmetro `pilar_input`.

Parâmetros

- `df_classificacao`: Um `data.frame` com as colunas `nome_uf`, `nota`, `classificacao`, e `pilar`.
- `pilar_input`: O valor do pilar que será utilizado para filtrar os dados e gerar a tabela.

Retorno

Retorna uma tabela `gt` com as seguintes características:

- Colunas UF, Nota, e Classificação.
- Formatação de números com 2 casas decimais, separador de milhar por ponto e separador decimal por vírgula.
- Coloração dos valores da coluna **Nota** usando a paleta “Blues”.
- Coloração da coluna **Classificação** com uma paleta personalizada, variando de vermelho (baixo) a verde (alto).

3.8 Renderização dos relatórios

Funções do script `report_utils.R`.

3.8.1 `display_results()`

Descrição

A função `display_results` gera uma mensagem textual que exibe os três principais ou os três piores resultados de um indicador específico, com base no valor do indicador. A função permite formatar os valores como porcentagens ou números, e customizar a exibição com o uso de `knitr::combine_words` para formatar a string de forma amigável.

Parâmetros

- `df_indicadores`: Um `data.frame` contendo os indicadores com as colunas `nome_uf`, `indicador`, e `valor`.
- `input_indicador`: O nome do indicador a ser filtrado nos dados.
- `perc`: Um valor lógico (`TRUE` ou `FALSE`) indicando se os valores devem ser exibidos como porcentagens (`TRUE`) ou números comuns (`FALSE`).
- `max`: Um valor lógico (`TRUE` ou `FALSE`) indicando se os 3 maiores valores devem ser exibidos (`TRUE`) ou os 3 menores valores (`FALSE`).

Retorno

Retorna uma string formatada que descreve os três melhores ou piores resultados para o indicador especificado, com os valores formatados conforme o parâmetro `perc`.

3.8.2 `display_stats()`

Descrição

A função `display_stats` calcula e exibe a média e o desvio padrão dos valores de um indicador específico. Os valores podem ser formatados como porcentagens ou números, dependendo do parâmetro `perc`.

Parâmetros

- `df_indicadores`: Um `data.frame` contendo os indicadores, com as colunas `indicador` e `valor`.
- `indicador_input`: O nome do indicador a ser analisado.
- `perc`: Um valor lógico (`TRUE` ou `FALSE`) que indica se os valores devem ser exibidos como porcentagens (`TRUE`) ou como números (`FALSE`).

Retorno

Retorna uma string formatada descrevendo a média e o desvio padrão dos valores do indicador, com os valores formatados conforme o parâmetro `perc`.

3.8.3 `display_ind_names()`

Descrição

A função `display_ind_names` gera uma string que combina o nome e a descrição de um indicador específico a partir de um `data.frame`. O resultado é formatado na forma de “indicador - descrição”.

Parâmetros

- `tbl_desc`: Um `data.frame` contendo as colunas `indicador` e `descricao`.
- `ind_input`: O nome do indicador para o qual se deseja obter a combinação de nome e descrição.

Retorno

Retorna uma string contendo o nome do indicador seguido de sua descrição.

3.8.4 `display_cor_results()`

Descrição

A função `display_cor_results` retorna o valor da correlação entre dois indicadores especificados. O valor é extraído de um `data.frame` de correlações e formatado para exibição com precisão de duas casas decimais.

Parâmetros

- `df_correlacao`: Um `data.frame` contendo as variáveis `var_a`, `var_b` e `cor`, onde `var_a` e `var_b` representam os indicadores e `cor` é o valor da correlação entre eles.

- `ind_a`: O nome do primeiro indicador (`var_a`).
- `ind_b`: O nome do segundo indicador (`var_b`).

Retorno

Retorna um valor numérico formatado representando a correlação entre os dois indicadores, com dois decimais.

3.8.5 `display_cor_table()`

Descrição

A função `display_cor_table` gera uma tabela de correlações entre variáveis de um determinado pilar, com p-valor inferior a 0.05. A tabela é formatada usando `knitr::kable`.

Parâmetros

- `df_correlacao`: Um `data.frame` contendo as colunas `var_a`, `var_b`, `cor`, e `p_value`, onde `var_a` e `var_b` são os indicadores e `cor` é o valor da correlação entre eles, e `p_value` é o p-valor da correlação.
- `pilar_input`: O pilar de interesse para filtrar as correlações.

Retorno

Retorna uma tabela em formato `kable` com as correlações significativas (p-valor < 0.05) entre os indicadores do pilar especificado.

3.8.6 `display_eda_table()`

Descrição

A função `display_eda_table` cria uma tabela que exibe as estatísticas descritivas de um indicador específico, com base em um conjunto de dados, como média, desvio padrão, mínimo, máximo, e mediana. A tabela é formatada e apresentada utilizando `knitr::kable`.

Parâmetros

- `df_eda`: Um `data.frame` contendo as colunas `indicador`, `valor`, e estatísticas como `media`, `desvio`, `min`, `max`, `mediana`.
- `ind_input`: O indicador para o qual as estatísticas descritivas serão filtradas.

Retorno

Retorna uma tabela `kable` com as estatísticas (média, desvio padrão, mínimo, máximo e mediana) formatadas para o indicador especificado.

3.8.7 `display_pca_direction()`

Descrição

A função `display_pca_direction` extrai a direção do componente principal (PC) especificado a partir de um conjunto de dados de estatísticas PCA. A função filtra os dados para remover os indicadores de “eigenvalue” e “variancia”, agrega os totais por linha, e então seleciona o valor correspondente ao componente principal especificado.

Parâmetros

- `df_stats`: Um `data.frame` contendo as estatísticas PCA, incluindo valores como “eigenvalue”, “variancia” e componentes principais.
- `pc_input`: O nome do componente principal de interesse (ex. “PC1”, “PC2”, etc.).

Retorno

Retorna o valor correspondente ao componente principal especificado para o indicador “Total”.

3.8.8 `display_pca_variance()`

Descrição

A função `display_pca_variance` extrai a variância associada a um componente principal (PC) específico a partir de um conjunto de dados de estatísticas PCA. A função filtra os dados para selecionar o indicador “variancia” e retorna o valor correspondente ao componente principal especificado.

Parâmetros

- `df_stats`: Um `data.frame` contendo as estatísticas PCA, incluindo a variância dos componentes principais.
- `pc_input`: O nome do componente principal de interesse (ex. “PC1”, “PC2”, etc.).

Retorno

Retorna o valor da variância associada ao componente principal especificado.

3.8.9 `display_notas()`

Descrição

A função `display_notas` exibe as três melhores ou piores notas de um determinado pilar. A função seleciona os estados com as três melhores ou piores notas, conforme indicado pelo parâmetro `pos`, e retorna uma frase concatenando os estados e suas respectivas notas.

Parâmetros

- `df_notas`: Um `data.frame` contendo as notas dos estados, incluindo a coluna `pilar` para filtrar os resultados por pilar.
- `pilar_input`: O pilar de interesse, utilizado para filtrar as notas.
- `pos`: Indica se as notas a serem exibidas são as melhores (“max”) ou as piores (“min”). O valor padrão é “max”.

Retorno

Retorna uma string formatada com os estados e suas respectivas notas, separados por vírgulas, com “e” antes do último estado.

3.8.10 `display_classificacao()`

Descrição

A função `display_classificacao` retorna os estados que pertencem a uma determinada classificação dentro de um pilar específico. A função filtra os dados por pilar e por classificação numérica e retorna os nomes dos estados, concatenados em uma única string.

Parâmetros

- `classificacao_df`: Um `data.frame` com informações sobre a classificação dos estados, incluindo as colunas `pilar` e `classificacao_numeric` para o filtro.
- `pilar_input`: O pilar de interesse, utilizado para filtrar os resultados.
- `class`: A classificação numérica de interesse, usada para filtrar os estados de acordo com a sua classificação.

Retorno

Retorna uma string com os estados que correspondem à classificação numérica especificada, separados por vírgulas, com “e” antes do último estado.

3.8.11 `calc_qnt_ind()`

Descrição

A função `calc_qnt_ind` calcula a quantidade de indicadores únicos presentes em um determinado pilar. A função filtra os dados para o pilar desejado e conta os indicadores únicos.

Parâmetros

- `df_indicadores`: Um `data.frame` contendo as informações sobre os indicadores, com a coluna `indicador` representando os diferentes indicadores.
- `pilar`: O pilar de interesse, utilizado para filtrar os indicadores.

Retorno

Retorna um número inteiro representando a quantidade de indicadores únicos no pilar especificado.

3.8.12 `calc_media_regiao()`

Descrição

A função `calc_media_regiao` calcula a média das classificações numéricas (`classificacao_numeric`) por região, utilizando as informações de classificação e a região associada a cada unidade federativa.

Parâmetros

- `df_class`: Um `data.frame` contendo as classificações numéricas, com a coluna `nome_uf` representando as unidades federativas e a coluna `classificacao_numeric` representando as classificações.
- `df_uf`: Um `data.frame` contendo as unidades federativas, com a coluna `nome_uf` e a coluna `regiao_uf` representando a região de cada unidade federativa.

Retorno

Retorna um `data.frame` contendo a média das classificações por região (`media_regiao`), agrupado por `regiao_uf`.

3.8.13 `display_class_regiao()`

Descrição

A função `display_class_regiao` formata os resultados da média das classificações por região, organizando as regiões pela média em ordem decrescente e retornando uma string que combina os nomes das regiões com suas respectivas médias.

Parâmetros

- `df_media`: Um `data.frame` contendo as médias das classificações por região, com as colunas `regiao_uf` representando as regiões e `media_regiao` representando as médias calculadas.

Retorno

Retorna uma string com os nomes das regiões e suas respectivas médias, organizados da maior para a menor média, separados por “e” (sem vírgulas ou “oxford comma”).

3.9 Renderização do dashboard

3.9.1 `data.R`

3.9.1.1 `arrange_classificacao_data()`

Descrição

A função `arrange_classificacao_data` organiza e transforma os dados de classificação, incorporando informações geográficas e criando representações visuais de estrelas com base nas classificações.

Parâmetros

- `df_class`: Um `data.frame` contendo dados de classificação com colunas como `nome_uf`, `pilar` e `classificacao_numeric`.
- `df_uf`: Um objeto `sf` contendo informações geográficas dos estados, incluindo uma coluna `name_state`.

Retorno

Retorna um objeto `sf` que combina as informações de classificação com dados geográficos, contendo as seguintes colunas:

- `name_region`: Região à qual o estado pertence.
- `nome_uf`: Nome do estado.
- `pilar`: Pilar associado à classificação.
- `star`: Representação da classificação em estrelas (*).
- `classificacao_numeric`: Classificação numérica.

3.9.1.2 `arrange_indicadores_data()`

Descrição

A função `arrange_indicadores_data` combina dados de indicadores com informações geográficas dos estados, padronizando nomes e regiões e assegurando que valores ausentes sejam substituídos.

Parâmetros

- `df_ind`: Um `data.frame` contendo os indicadores, incluindo colunas como `cod_uf`, `indicador` e `valor`.
- `df_uf`: Um objeto `sf` contendo informações geográficas dos estados, incluindo uma coluna `code_state`.

Retorno

Retorna um objeto `sf` com as seguintes colunas:

- `nome_uf`: Nome do estado.
- `regiao_uf`: Região à qual o estado pertence, com nomes padronizados.
- `indicador`: Indicador associado.
- `valor`: Valor do indicador, com valores ausentes substituídos por 1.

3.9.1.3 `join_desc_data()`

Descrição

A função `join_desc_data` realiza a junção de um conjunto de dados de indicadores com uma tabela de descrições, garantindo que os indicadores estejam no mesmo formato (maiúsculas) para a correspondência.

Parâmetros

- `df_ind`: Um `data.frame` contendo indicadores e suas informações associadas.
- `df_desc`: Um `data.frame` contendo descrições dos indicadores, com uma coluna `indicador`.

Retorno

Retorna um `data.frame` com as colunas de `df_ind` combinadas com as descrições de `df_desc`.

3.9.2 `mapa.R`

3.9.2.1 `plot_classificacao_leaflet()`

Descrição

A função `plot_classificacao_leaflet` cria um mapa interativo utilizando `leaflet`, onde as unidades federativas são coloridas com base em sua classificação (`star`) para um pilar específico.

Parâmetros

- `df_classificacao`: Um `data.frame` espacial (`sf`) contendo as classificações das unidades federativas e o pilar correspondente.
- `pilar_input`: Uma string indicando o pilar a ser exibido no mapa.

Retorno

Retorna um objeto `leaflet` representando o mapa interativo.

3.9.2.2 `plot_leaflet_geral()`

Descrição

Cria um mapa interativo utilizando o pacote `leaflet`, exibindo a classificação média das unidades federativas em uma escala de 1 a 5.

Parâmetros

- `df_classificacao`: Um `data.frame` espacial (`sf`) contendo as classificações das unidades federativas.

Retorno

Retorna um objeto `leaflet` com o mapa interativo representando a classificação média das unidades federativas.

3.9.2.3 `plot_indicadores_leaflet()`

Descrição

Cria um mapa interativo utilizando o pacote `leaflet`, representando os valores de um indicador específico para as unidades federativas.

Parâmetros

- `df_indicadores`: Um `data.frame` espacial (`sf`) contendo os valores dos indicadores por UF.
- `ind_input`: O nome do indicador a ser exibido no mapa.

Retorno

Retorna um objeto `leaflet` com o mapa interativo representando os valores do indicador para as unidades federativas.

3.9.3 `plots.R`

3.9.3.1 `plot_radar_class()`

Descrição

Gera um gráfico de radar utilizando o `plotly`, representando a classificação de uma unidade federativa (UF) em diferentes pilares.

Parâmetros

- `class_data`: Um `data.frame` espacial (`sf`) contendo as classificações das UFs por pilar.
- `uf`: O nome da unidade federativa a ser exibida no gráfico.

Retorno

Retorna um objeto `plotly` com o gráfico de radar representando as classificações da UF nos diferentes pilares.

3.9.4 `table.R`

3.9.4.1 `make_class_dt()`

Descrição

Cria uma tabela interativa utilizando `DT::datatable`, exibindo informações de classificação das unidades federativas (UFs) em um pilar específico.

Parâmetros

- `sf_class`: Um objeto `sf` contendo as classificações das UFs por pilar.
- `pilar_input`: Pilar selecionado para filtrar as classificações.

Retorno

Retorna um objeto `datatable` com coloração condicional baseada nas estrelas de classificação.

3.9.4.2 `make_ind_dt()`

Descrição

Cria uma tabela interativa com as informações de um indicador específico por unidade federativa (UF), utilizando `DT::datatable`. A tabela inclui estilos condicionais para destacar os valores de cada UF.

Parâmetros

- `sf_ind`: Um objeto `sf` contendo os dados dos indicadores por UF.
- `ind_input`: Indicador selecionado para exibição.

Retorno

Retorna um objeto `datatable` formatado, com coloração baseada nos valores do indicador e números ajustados para percentual ou número com separadores.

3.9.4.3 `make_results_gt()`

Descrição

Gera uma tabela estilizada usando `gt` para exibir os resultados de indicadores de uma unidade federativa (UF) específica, com informações detalhadas sobre a descrição e unidade de medida.

Parâmetros

- `df_desc_data`: `DataFrame` com os dados de indicadores, incluindo descrição e valores.
- `ind_input`: Indicador selecionado para exibição.
- `uf`: Unidade Federativa (UF) de interesse.

Retorno

Uma tabela formatada com colunas: Indicador, Descrição [Unidade], e Resultado, com números e percentuais ajustados e estilos personalizados.

3.9.4.4 `make_gt_bench()`

Descrição

Cria uma tabela estilizada com a classificação por pilar de uma unidade federativa (UF) específica, destacando os melhores resultados (`****`) por pilar.

Parâmetros

- `sf_class`: DataFrame espacial com as classificações por pilar.
- `uf`: Unidade Federativa (UF) de interesse.

Retorno

Uma tabela formatada usando `gt` com as colunas:

- Pilar: Nome do pilar analisado.
- Classificação: Nota atribuída à UF no respectivo pilar (estilizada por cores).
- Melhores resultados (`****`): UFs que alcançaram a melhor classificação no respectivo pilar.

Capítulo 4

Publicação

O presente capítulo apresenta as instruções dos últimos passos do projeto - a publicação dos *outputs* de todo o código.

4.1 Arquivos dos relatórios

Os projetos do Quarto Markdown inseridos na pasta `report` geram dois arquivos de saída dentro da pasta `report/[entrega]/docs`: o mesmo relatório no format PDF e DOCX. O meio de publicação destes arquivos fica a critério do usuário.

4.2 Publicação do dashboard

O dashboard foi hospedado com o `shinyapps`¹ - uma plataforma específica para realizar o deploy de aplicativos desenvolvidos com o `{shiny}`. Para isso, é necessário a instalação do pacote `{rsconnect}` (já instalado ao executar `renv::restore()`) e a utilização da IDE RStudio².

O primeiro passo consiste em fazer o login na plataforma com as seguintes credenciais:

login: `pedro.borges@onsv.org.br`

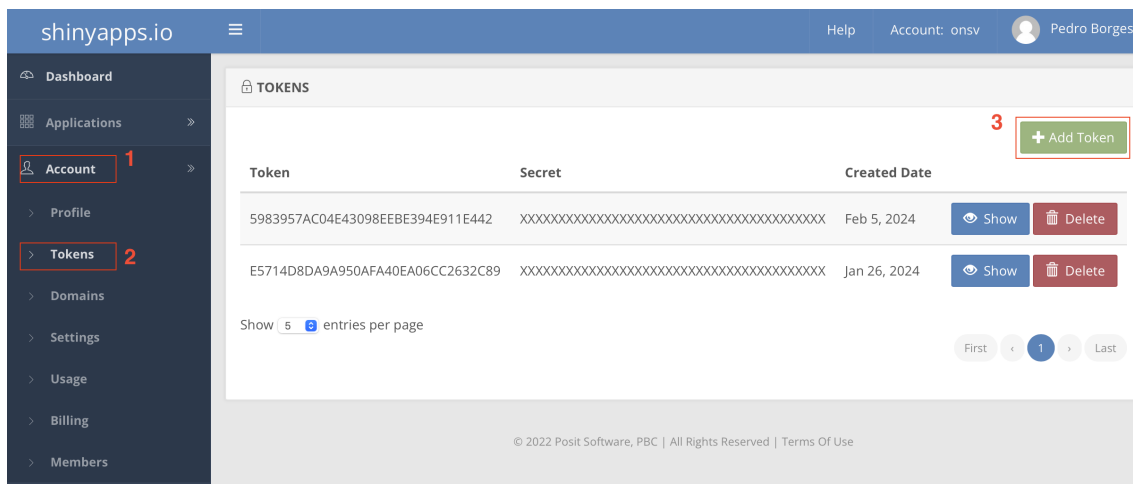
senha: `Onsv@2024`

Em seguida, é necessário gerar um token para conectar o seu ambiente de trabalho com a conta no shinyapps. Para isso, com o login efetuado, acesse o menu Account -> Tokens -> Add Token, conforme indicado na Figura 4.1.

¹<https://shinyapps.io/>

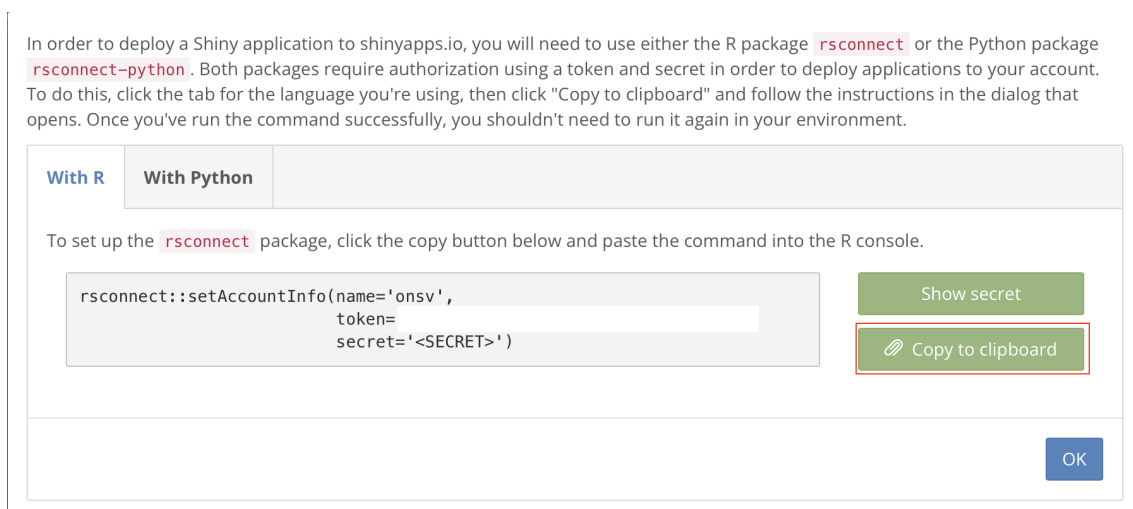
²É possível realizar o deploy sem o RStudio, porém deixa o processo um pouco mais demorado.

Figura 4.1: Passo 01 - Criação do token



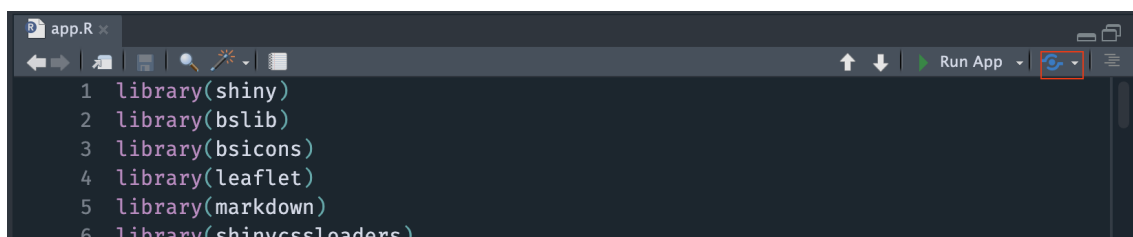
Com o token gerado, clique para ver o seu conteúdo e depois clique em “Copy to clipboard”, para copiar o comando (Figura 4.2). Em seguida, cole esse comando no console do RStudio e execute-o. Agora o seu RStudio está conectado com o shinyapps.

Figura 4.2: Passo 02 - Comando de conexão



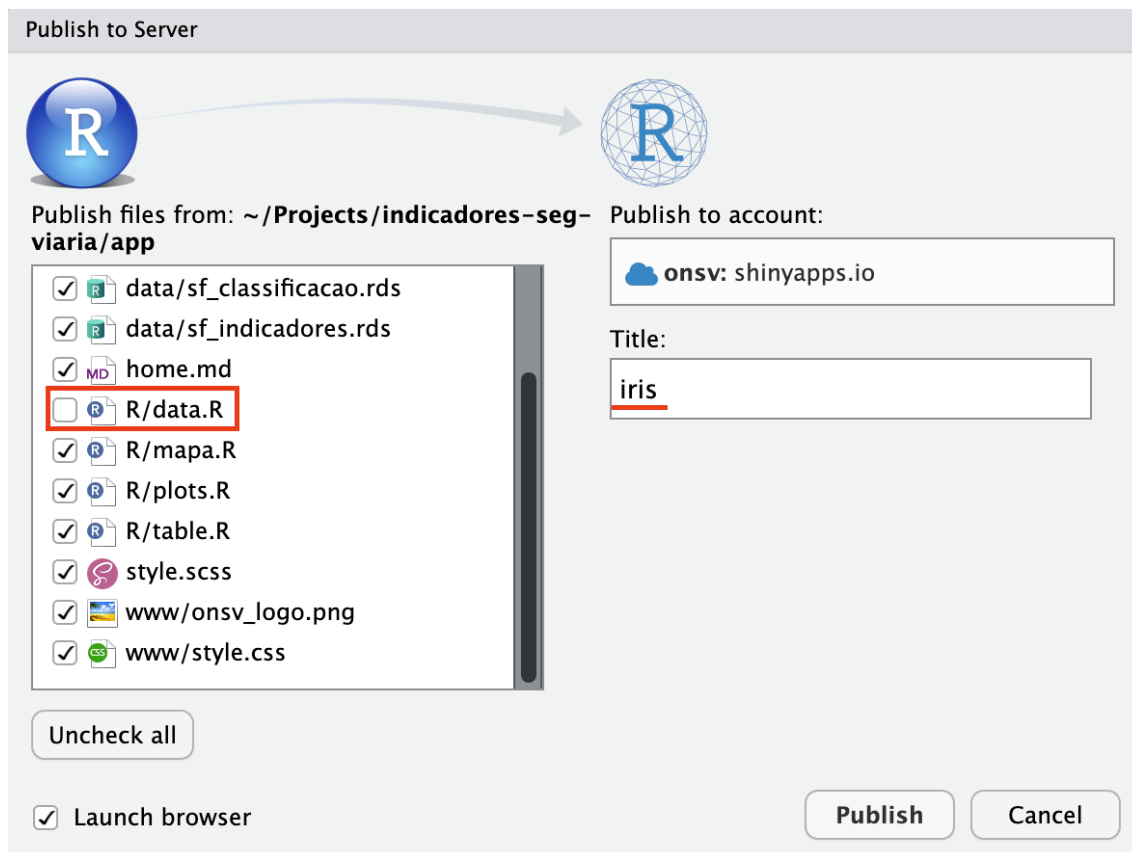
Para publicar o dashboard, é necessário clicar no botão “Publish”, ao lado de “Run App”, conforme indicado na Figura 4.3.

Figura 4.3: Passo 03 - Botão de publicação



Ao clicar em “Publish”, uma tela similar ao da Figura 4.4 irá abrir. Aqui é importante selecionar todos os arquivos, com exceção de R/data.R. Em seguida, mude o título para “iris” e utilize o botão “Publish” para executar a publicação do dashboard.

Figura 4.4: Passo 04 - Paine de publicação



Apos clicar em “Publish”, o processo será executado. Geralmente é um processo um pouco demorado e é importante que ele não seja interrompido.

Referências

- Chang, W., Cheng, J., Allaire, J., Sievert, C., Schloerke, B., Xie, Y., Allen, J., McPherson, J., Dipert, A., e Borges, B. (2024). *shiny: Web Application Framework for R*. <https://CRAN.R-project.org/package=shiny>
- Landau, W. M. (2021). The targets R package: a dynamic Make-like function-oriented pipeline toolkit for reproducibility and high-performance computing. *Journal of Open Source Software*, 6(57), 2959. <https://doi.org/10.21105/joss.02959>
- R Core Team. (2024). *R: A Language and Environment for Statistical Computing* [Manual]. R Foundation for Statistical Computing.
- Ushey, K., e Wickham, H. (2024). *renv: Project Environments*. <https://CRAN.R-project.org/package=renv>